SKETCHPAD

The Figures code, commented

Phases 5 to 8: figures, layers, discs, moving and turning, welds

Built 27 October 2026 from claude/docs at 4392c1d.
Twelve files, in reading order, with an introduction.

Contents

BEFORE THE CODE

How to read this 3
The files at a glance 4
Two actions, end to end 5
JavaScript you will meet 6

THE FILES, IN READING ORDER

figureModel.js The figure as plain data 7
figures.js Entry point; where all the state lives 9
figuresNav.js Subtabs, the left-hand controls 14
figureLayers.js Insert, draw, select and hide layers 22
drawTransform.js Moving and turning: the math 32
figurePoints.js Discs: data, finding, drawing 36
figurePointer.js The mouse and keys in points mode 43
figureCentroid.js Move and turn a whole layer 49
figureWelds.js Welds 55
figuresCommands.js Commands panel and caption menu 65
figuresDisplay.js The saved-figure category list 69
figuresState.js Navigation bookkeeping 71

APPENDIX

interactor.js Handle dragging, shared with the Draw tab (not re-commented) 73

How to read this

These are the source files of the Figures tab, Phases 5 to 8, with comments added for reading. The comments changed; the code did not. Each file was checked by stripping its comments and comparing the result, byte for byte, with the file before the comments were added.

Comments are set in dark green so they read as text and the code stands apart from them. Every file opens with a header that says what the file is for, what it must not do, and what it exports; read the header first and the code after.

The files are in reading order, not alphabetical. The order follows the data: first what a figure *is* (`figureModel.js`), then where it lives and how you get to it, then how its layers are drawn, then the discs on those layers, then the two ways to move things, then welds, which tie the rest together. The last three files are small and can be read in any order.

Every file entry in the contents is a link, and every file starts with a link back to the contents. The PDF's bookmark panel lists the same entries.

The files at a glance

File	What it does	Lines
`ui/figures/figureModel.js`	The figure as plain data	94
`ui/figures/figures.js`	Entry point; where all the state lives	278
`ui/figures/figuresNav.js`	Subtabs, the left-hand controls	452
`ui/figures/figureLayers.js`	Insert, draw, select and hide layers	625
`draw/drawTransform.js`	Moving and turning: the math	194
`ui/figures/figurePoints.js`	Discs: data, finding, drawing	388
`ui/figures/figurePointer.js`	The mouse and keys in points mode	354
`ui/figures/figureCentroid.js`	Move and turn a whole layer	372
`ui/figures/figureWelds.js`	Welds	580
`ui/figures/figuresCommands.js`	Commands panel and caption menu	218
`ui/figures/figuresDisplay.js`	The saved-figure category list	123
`ui/figures/figuresState.js`	Navigation bookkeeping	84
`ui/interactor.js`	Handle dragging, shared with the Draw tab (not re-commented)	311

Two actions, end to end

The files make most sense once you have followed one thing the user does all the way through.

Dragging a handle that is welded to another object

1. `interactor.js` sees the press on a handle of the active object and starts a drag.
2. On each mouse move it calls `moveHandle()`, which first calls the layer's `beforeMove` (set up in `insertLayer()`, `figureLayers.js`). That takes a snapshot of where this layer's welded discs are (`snapshotWeldMembers`, `figureWelds.js`).
3. The handle's position is written into `params.points`, mapped from the canvas to the layer's own coordinates through its transform (`drawTransform.js`).
4. `onUpdate` runs the object's own `update()` and then `redrawFigure()`. Every layer redraws; `refreshPoints()` (`figurePoints.js`) moves each anchored disc to its fraction along its arm, so the welded disc on this layer has now moved.
5. `afterMove` calls `propagateAfterChange()`. It compares the snapshot with where the discs are now, and for each one that moved, it moves the weld partners on other layers by the same amount, choosing whether to move one handle, both ends of an arm, or the whole layer.
6. If anything followed, one more `refreshPoints()` and `redrawFigure()` show the result.

A click in points mode

1. `figurePointer.js` records where the left button went down, and whether that was on a handle (then the interactor owns the press) or on a free disc (then a drag may move it).
2. On release, if it was a click: a disc of the active layer there is selected; failing that, an infrastructure point within the hit radius is numbered as an anchored disc (`findCandidate`, `addAnchoredPoint`); failing that, a free disc is placed.
3. `redrawFigure()` draws it.

A right-click goes to `toggleWeldAt()` in `figureWelds.js` instead, and only there.

JavaScript you will meet

The code is plain on purpose, but modern JavaScript has some short forms that recur. Each is explained in a comment where it first appears in a file; this is the whole list in one place.

Written	Means
`a?.b`	a.b, or undefined if a is missing (instead of an error)
`a || b`	a, unless a is empty (null, undefined, 0, "", false); then b
`a ?? b`	a, unless a is null or undefined; then b. Unlike ||, keeps a 0.
`!!a`	a turned into a plain true or false
`a ? b : c`	if a then b else c, as an expression
`(x) => x + 1`	a small function written in place ("arrow function")
`` `text ${x}` ``	a string with x filled in (backquotes)
`const { a, b } = obj`	a = obj.a and b = obj.b, in one line
`for (const [a, b] of list)`	walk a list of pairs, naming both halves
`{ ...obj }`, `[x, ...list]`	a copy of obj; a new list with x first, then list
`for (k in obj)` / `for (v of list)`	walk an object's keys / a list's values
`new Set()`, `new Map()`	a collection of unique values / a lookup table
`list.filter(...)`, `.map(...)`, `.find(...)`, `.every(...)`	keep the matching items / transform each / the first match / true if all match
`structuredClone(obj)`	a deep copy: nothing nested is shared
`Object.create(entry)`	a new object that falls back on entry for anything it lacks; how each layer gets its own params but shares the registry object's functions
`async` / `await`	wait for a slow answer (a file fetch) without freezing the page
`import("...")` in a function	load a module only when it is first needed
`new AbortController()`, `{ signal }`	every listener registered with the signal is removed by one `abort()`
a function that uses a variable from around it	a "closure": it keeps that variable alive and private (each layer's `weldSnapshot`)

figureModel.js

ui/figures/figureModel.js · 94 lines

THE FIGURE AS PLAIN DATA

← Contents

```
/* figureModel.js
   ============================================================
   Figures Tab -- The Figure Data Model
   ============================================================
   Role:
     Plain data and small helper functions for one figure. No DOM,
     no imports, no drawing. A figure lives in
     uiState.figures.tabs[tabId].figure.

   A figure is plain data, nothing but numbers, strings, arrays and
   plain objects. That is on purpose: it can be copied with
   structuredClone(), written to a file with JSON.stringify() (Save
   As, Phase 10), and compared or snapshotted (a possible undo). The
   live drawing objects (instances) that draw each layer are kept
   beside it in tabState.instances, never inside it.

   A figure, field by field:
     name        -- display name, "Untitled 1" until Save As names it
     category    -- null until Save As picks one
     nextPoint   -- the next point number to hand out. Numbers are
                    never reused, even after a point is deleted, so
                    a number names one disc for good (welds refer to
                    discs by number).
     points      -- every disc of every layer, in one flat array.
                    Two shapes (figurePoints.js has the details):
                      { n, kind: "anchored", layer, arm, t, x, y, dx, dy }
                      { n, kind: "free",     layer, x, y, dx, dy, ox, oy }
                    An anchored disc sits at fraction t along an
                    infrastructure arm of its layer's object and
                    follows it. A free disc sits where it was put,
                    stored in its layer's object space (ox, oy).
     layers      -- one entry per inserted object (figureLayers.js):
                      { kind, id, visible, color, params, transform }
                    A layer's number is its index plus one and never
                    changes; layers are never removed (hidden instead).
     drawOrder   -- layer indices, top of the stack first. Kept
                    separate from layers so restacking never
                    renumbers anything.
     activeLayer -- index into layers, -1 when there are none. Only
                    the active layer can be edited.
     welds       -- added by figureWelds.js on the first weld (absent
                    until then):
                      [ { members: [ { layer, n }, ... ] }, ... ]
                    Each entry ties together discs that must stay
                    on the same spot.

   Exports:
     createEmptyFigure(name)
     nextUntitledName(tabs)
     nextFigureTabId(tabs)
   ============================================================ */

// A new, empty figure. Called by openNewFigureTab() in figuresNav.js.
// "welds" is not listed here; figureWelds.js creates it the first
// time two discs are welded, and every reader allows for it being
// missing (figure.welds || []).
export function createEmptyFigure(name) {
  return {
```

```
    name,               // shorthand for  name: name
    category:    null,
    nextPoint:   1,
    points:      [],
    layers:      [],
    drawOrder:   [],   // layer indices, top of the stack first
    activeLayer: -1
  };
} // end createEmptyFigure


// Smallest "Untitled N" not used by an open figure tab.
// tabs is uiState.figures.tabs: an object keyed by tab id, one of which
// ("tab-categories") has no figure, hence the "if (f)".
export function nextUntitledName(tabs) {
  const used = new Set();          // a Set holds each value once; .has() asks
  for (const id in tabs) {         // "for ... in" walks an object's keys
    const f = tabs[id].figure;
    if (f) used.add(f.name);
  }
  let n = 1;
  while (used.has("Untitled " + n)) n++;
  return "Untitled " + n;
} // end nextUntitledName


// Smallest "tab-figure-N" id not used in the tabs object. A closed
// tab's id becomes free again, unlike a point number: tab ids are
// only screen bookkeeping and nothing else refers to them.
export function nextFigureTabId(tabs) {
  let n = 1;
  while (tabs["tab-figure-" + n]) n++;
  return "tab-figure-" + n;
} // end nextFigureTabId

// end figureModel.js
```

figures.js

ui/figures/figures.js · 278 lines

ENTRY POINT; WHERE ALL THE STATE LIVES

← Contents

```
/* figures.js
   ============================================================
   Figures Tab -- Public Entry Point and Lifecycle
   ============================================================
   Role:
     Public entry point for the Figures tab. Owns exactly
     three things:

       1. FiguresTabSpec -- the object consumed by setUI.js to
          drive tab activation (init / restore / save).

       2. Lifecycle functions -- initFiguresTab().

       3. UI region setup -- setFiguresAction(), setFiguresText(),
          setFiguresSketchpad().

   What does NOT live here:
     * Subtab construction and navigation    -> figures/figuresNav.js
     * Category loading and figure loading   -> figures/figuresDisplay.js
     * Shared module-level state variables   -> figures/figuresState.js
     * The figure data model                 -> figures/figureModel.js
     * Commands button and caption bar       -> figures/figuresCommands.js
     * Layers: insert, draw, select, hide    -> figures/figureLayers.js
     * Discs (numbered points)               -> figures/figurePoints.js
     * Mouse and keys in points mode         -> figures/figurePointer.js
     * Moving and turning a whole layer      -> figures/figureCentroid.js
     * Welds                                 -> figures/figureWelds.js
     * The layer transform (the math)        -> draw/drawTransform.js

   How the files depend on each other (an arrow means "imports"):

     figures.js      -> figuresCommands, figuresState, figuresNav, figuresDisplay
     figuresNav      -> figuresState, figuresCommands, figureLayers,
                        figurePointer, figureCentroid, figureModel
     figuresCommands -> figureLayers, figurePoints
     figuresDisplay  -> figuresState
     figureLayers    -> figurePoints, figurePointer, figureCentroid, figureWelds
     figurePointer   -> figurePoints, figureWelds
     figureCentroid  -> figureWelds
     figureWelds    <-> figureLayers, figurePoints   (each pair imports the
                                                      other; safe, see
                                                      figureWelds.js)
     figureModel, figuresState: no imports at all.

   Where the figure's state lives:
     uiState.figures = {
       activeSubtab: "tab-categories" or "tab-figure-N",
       tabs: {
         "tab-categories": { type: "categories" },
         "tab-figure-1":   { type: "figure", figure, instances, mode,
                             selectedPoint, centroidSelected, preview }
       },
       hitRadius: 5,     how close a click must be to pick a point (px)
       saved: null       reserved for the TabSpec save/restore pattern
     }
     Because it is all in uiState, a figure survives switching to
     another main tab and back with no saving needed (CLAUDE.md:
     "state is written through immediately").
```

```
   ============================================================ */

import { setCommandsButtonLabel }    from "/ui/uiUtilities.js";
import { manifest }                  from "/ui/manifest.js";
import {
  setFiguresCaption,
  wireFiguresCommandsButton
}                                    from "/ui/figures/figuresCommands.js";
import {
  resetFiguresState,
  setCurrentTabId
}                                    from "/ui/figures/figuresState.js";
import {
  setFiguresSubtabs,
  renderSubtabs,
  switchToCategories,
  switchToFigureTab,
  openNewFigureTab
}                                    from "/ui/figures/figuresNav.js";
import { loadFiguresCategories }     from "/ui/figures/figuresDisplay.js";


/* ============================================================
   FiguresTabSpec
   ------------------------------------------------------------
   setUI.js activates every main tab through one of these. Its
   decision order: if the tab needs an update, init(true); else if
   it was saved, restore; else init(false). Figures keeps its state
   in uiState (see above), so save() has nothing extra to hand back,
   uiState.figures.saved stays null, and setUI therefore always calls
   init(). There is no restore: init() itself shows whatever subtab
   was active last.
   ============================================================ */
export const FiguresTabSpec = {
  theme: "theme-figures",

  init: initFiguresTab,
  save: () => ({}),

  action:    setFiguresAction,
  subtabs:   setFiguresSubtabs,
  caption:   setFiguresCaption,
  text:      setFiguresText,
  sketchpad: setFiguresSketchpad
};


/* ============================================================
   initFiguresTab(restored)
   ------------------------------------------------------------
   Called every time the Figures tab is shown. Rebuilds the tab's
   screen regions and returns to the subtab that was active.
   ============================================================ */
export function initFiguresTab(restored = false) {

  // A guard only: uiState.js already declares uiState.figures with these
  // same values (CLAUDE.md: every global value is declared in its state
  // object), so this block does not normally run. "!uiState.figures" is
  // true only if that declaration were ever removed.
  if (!uiState.figures) {
    uiState.figures = {
      activeSubtab: "tab-categories",
      tabs: {
        "tab-categories": { type: "categories" }
      },
```

```
      hitRadius: 5,
      saved: null
    };
  }

  setCommandsButtonLabel("Figures Commands");
  wireFiguresCommandsButton();

  setFiguresAction();
  setFiguresSubtabs();
  setFiguresCaption();
  setFiguresText();
  setFiguresSketchpad();

  loadFiguresCategories();

  renderSubtabs();

  const activeId = uiState.figures.activeSubtab;

  // Go back to the figure that was showing, if it still exists.
  if (activeId && activeId !== "tab-categories") {
    if (uiState.figures.tabs[activeId]) {
      switchToFigureTab(activeId);
    } else {
      switchToCategories();
    }
  } else {
    switchToCategories();
  }

  // Register with the manifest system once, so a Rebuild & Validate
  // re-runs this init. A function is an object in JavaScript, so it can
  // carry a property (_registered) that remembers it was done.
  if (!initFiguresTab._registered) {
    manifest.register("figures", () => initFiguresTab(false));
    initFiguresTab._registered = true;
  }

} // end initFiguresTab


/* ============================================================
   UI Region Setup Functions
   ============================================================ */

/* ------------------------------------------------------------
   setFiguresAction()
   Builds the Figures controls in the permanent #button div, under
   the Commands button, in this order:
     Show All checkbox      (figure subtabs only)
     Points Mode checkbox   (figure subtabs only)
     Create Figure button   (Categories subtab only)
     Insert Object list     (figure subtabs only)
   They are all built here once and shown or hidden per subtab by
   updateFigureButtons() in figuresNav.js, which also wires the two
   checkboxes and fills the list. Every element carries the class
   "tab-extra" so clearDivs() removes them when another main tab
   is shown (CLAUDE.md: no ad hoc divs; #button is permanent).
   ------------------------------------------------------------ */
function setFiguresAction() {
  /* Append Create Figure button into #button below Commands.
     Guard against double-adding on restore.                  */
  if (document.getElementById("createFigureButton")) return;

```

```
    const buttonDiv = document.getElementById("button");
    if (!buttonDiv) return;

    // Tagged "tab-extra": #button is permanent, so clearDivs() removes
    // everything tagged this way on each tab switch.
    // Show All (preview) toggle, figure tabs only; wired in figuresNav.js.
    const brP = document.createElement("br");
    brP.id = "figureBrPreview";
    brP.className = "tab-extra";
    buttonDiv.appendChild(brP);

    const prevLabel = document.createElement("label");
    prevLabel.id        = "figurePreviewLabel";
    prevLabel.className = "tab-extra";
    prevLabel.title     = "Preview: every layer at full strength, hidden layers included, as the final
drawing will look";

    const prevCheck = document.createElement("input");
    prevCheck.type = "checkbox";
    prevCheck.id   = "figurePreviewCheck";
    prevLabel.appendChild(prevCheck);
    prevLabel.appendChild(document.createTextNode(" Show All"));
    buttonDiv.appendChild(prevLabel);

    // Points Mode checkbox (figure tabs only; wired in figuresNav.js). A label
    // wraps the checkbox so the text is clickable too.
    const brM = document.createElement("br");
    brM.id = "figureBrMode";
    brM.className = "tab-extra";
    buttonDiv.appendChild(brM);

    const modeLabel = document.createElement("label");
    modeLabel.id        = "figureModeLabel";
    modeLabel.className = "tab-extra";
    modeLabel.title     = "Show only points. Click to place a point on the active layer; Delete
removes the selected one. The object's handles still drag.";

    const modeCheck = document.createElement("input");
    modeCheck.type = "checkbox";
    modeCheck.id   = "figureModeCheck";
    modeLabel.appendChild(modeCheck);
    modeLabel.appendChild(document.createTextNode(" Points Mode"));
    buttonDiv.appendChild(modeLabel);

    const br = document.createElement("br");
    br.id = "figureBr1";
    br.className = "tab-extra";
    buttonDiv.appendChild(br);

    const btn = document.createElement("button");
    btn.id          = "createFigureButton";
    btn.className   = "cmdButton tab-extra";
    btn.type        = "button";
    btn.textContent = "Create Figure";
    btn.onclick     = openNewFigureTab;   // the function itself, not a call: run on click
    buttonDiv.appendChild(btn);

    // Insert Object: a list of every registry object. Filled and enabled
    // by updateFigureButtons() in figuresNav.js. Create Figure shows only
    // on the Categories tab; Insert Object only on a figure tab.
    const br2 = document.createElement("br");
    br2.id = "figureBr2";
    br2.className = "tab-extra";
    buttonDiv.appendChild(br2);

```

```
  const sel = document.createElement("select");
  sel.id        = "figureInsertSelect";
  sel.className = "cmdButton tab-extra";
  sel.title     = "Insert any registry object as a new layer";
  buttonDiv.appendChild(sel);

} // end setFiguresAction

// The #text region holds the category list. A placeholder until
// loadFiguresCategories() fills it.
function setFiguresText() {
  const el = document.getElementById("text");
  if (el) el.innerHTML = "Loading categories...";
} // end setFiguresText

// Moves the one shared drawing canvas (window.drawCanvas, used by
// every tab) into this tab's #sketchpad region, unless it is already
// there.
function setFiguresSketchpad() {
  const sketchpad = document.getElementById("sketchpad");
  const canvas    = window.drawCanvas;

  if (sketchpad && canvas && !sketchpad.contains(canvas)) {
    sketchpad.appendChild(canvas);
    canvas.style.display = "block";
  }
} // end setFiguresSketchpad
```

figuresNav.js

ui/figures/figuresNav.js · 452 lines

SUBTABS, THE LEFT-HAND CONTROLS

← Contents

```
/* figuresNav.js
   ============================================================
   Figures Tab -- Subtab Construction and Navigation
   ============================================================
   Role:
     Owns everything related to building the Figures subtab bar,
     switching between tabs, and managing tab lifecycle (open/close).

   Architectural rules:
     * Does NOT own the TabSpec, init(), restore(), or save().
       Those live in figures.js.
     * Does NOT load categories. That lives in figuresDisplay.js.
     * Does NOT build caption bars or menu items.
       Those live in figuresCommands.js.
     * Reads figuresRegistry via getters from figuresState.js.

   The subtab bar:
     [Categories] [Untitled 1 x] [Untitled 2 x] ...
     Categories is always first and cannot be closed. Each figure
     subtab shows its figure's name and a close mark. The data for
     each subtab is uiState.figures.tabs[tabId]; the active one is
     uiState.figures.activeSubtab.

   What one figure subtab holds (tabState, uiState.figures.tabs[tabId]):
     type              "figure"
     figure            the plain data (figureModel.js)
     instances         one live drawing object per layer, same index
                       as figure.layers (figureLayers.js)
     mode              "lines" or "points"
     selectedPoint     the number of the selected disc, or null
     centroidSelected  true while the active layer's diamond is selected
     preview           true while Show All is ticked
   All but figure are view state: they would not be saved with the
   figure.

   Exports:
     setFiguresSubtabs()
     renderSubtabs()
     updateFigureButtons()
     switchToCategories()
     switchToFigureTab(tabId)
     openNewFigureTab()
     closeTab(tabId)
   ============================================================ */

import { setCurrentTabId }     from "./figuresState.js";
import { setFiguresCaption }   from "./figuresCommands.js";
import {
  listInsertableObjects,
  insertLayer,
  setLayerVisible,
  redrawFigure,
  renderLayerPanel
}                              from "./figureLayers.js";
import { deactivatePointsInput } from "./figurePointer.js";
import { deactivateCentroid }    from "./figureCentroid.js";
import {
  createEmptyFigure,
```

```
  nextUntitledName,
  nextFigureTabId
}                              from "./figureModel.js";


/* ============================================================
   setFiguresSubtabs()
   ------------------------------------------------------------
   Puts an empty tab bar (a Bootstrap "nav-tabs" list) into #subtabs.
   renderSubtabs() fills it.
   ============================================================ */
export function setFiguresSubtabs() {

  const el = document.getElementById("subtabs");
  if (!el) throw new Error("setFiguresSubtabs: #subtabs not found");

  el.innerHTML = "";

  const bar = document.createElement("ul");
  bar.className = "nav nav-tabs figure-subtabs";
  el.appendChild(bar);

} // end setFiguresSubtabs


/* ============================================================
   renderSubtabs()
   ------------------------------------------------------------
   Rebuilds the whole bar from uiState.figures.tabs. Called on every
   switch, open and close; rebuilding is simpler than patching and
   the bar is small.
   ============================================================ */
export function renderSubtabs() {

  const bar = document.querySelector("#subtabs ul");
  if (!bar) return;

  bar.innerHTML = "";

  const activeTabId = uiState.figures.activeSubtab;

  // Categories tab (always present)
  const catLi  = document.createElement("li");
  catLi.className = "nav-item";

  const catBtn = document.createElement("button");
  catBtn.className  = "nav-link";
  if (activeTabId === "tab-categories") catBtn.classList.add("active");
  catBtn.textContent = "Categories";
  catBtn.onclick     = () => switchToCategories();

  catLi.appendChild(catBtn);
  bar.appendChild(catLi);

  // Figure tabs
  for (const tabId in uiState.figures.tabs) {
    if (tabId === "tab-categories") continue;

    const tab = uiState.figures.tabs[tabId];

    const li  = document.createElement("li");
    li.className = "nav-item";

    const btn = document.createElement("button");
    btn.className = "nav-link";
```

```
    if (activeTabId === tabId) btn.classList.add("active");

    // "a?.b" reads b only if a exists (otherwise gives undefined instead
    // of an error); "x || y" gives y when x is empty. So: the figure's
    // name, else a title, else the tab id without its "tab-" prefix.
    const label = document.createElement("span");
    label.textContent = tab.figure?.name || tab.title || tabId.replace("tab-", "");
    btn.appendChild(label);

    const close = document.createElement("span");
    close.textContent   = " ×";
    close.style.cursor  = "pointer";
    close.style.marginLeft = "5px";
    close.onclick = (e) => {
      // The close mark sits inside the tab button. Without this the click
      // would also reach the button and switch to the tab being closed.
      e.stopPropagation();
      closeTab(tabId);
    };
    btn.appendChild(close);

    btn.onclick = () => switchToFigureTab(tabId);

    li.appendChild(btn);
    bar.appendChild(li);
  }

  updateFigureButtons();

} // end renderSubtabs


/* ============================================================
   addCaptionTools(tabId)
   ------------------------------------------------------------
   Puts the figure's tools in the gray caption bar (same row as the
   title and the "v" menu button). Call after setFiguresCaption(),
   which rebuilds the bar.

     middle : Line and Parabola shortcut buttons
     right  : "Layer N . Name" and the Show checkbox (filled in by
              figureLayers.js), just left of the "v" button

   The full registry list is the Insert Object list in the left
   command area (see updateFigureButtons).
   ============================================================ */
function addCaptionTools(tabId) {

  const caption = document.getElementById("caption");
  const btnRow  = caption?.querySelector(".caption-buttons");
  if (!caption || !btnRow) return;

  // ---- middle: Line, Parabola ---------------------------------
  const mid = document.createElement("div");
  mid.id = "figureCaptionTools";
  mid.style.cssText = "display:flex; gap:8px; flex-grow:1; justify-content:center;";

  // [button text, registry key, tooltip]
  const tools = [
    ["Line",     "drawLine", "Insert a line as a new layer"],
    ["Parabola", "parabola", "Insert a parabola (curve-stitch between two arms) as a new layer"]
  ];

  // "const [a, b, c] of list" unpacks each three-element entry into names.
  for (const [label, key, tip] of tools) {
```

```
    const b = document.createElement("button");
    b.type = "button";
    b.id = "figureInsert_" + key;     // the tests find the buttons by these ids
    b.textContent = label;
    b.title = tip;
    b.onclick = () => insertLayer(tabId, key);
    mid.appendChild(b);
  }
  caption.insertBefore(mid, btnRow);

  // ---- right: active layer name and Show checkbox ----------------
  const info = document.createElement("span");
  info.style.cssText = "display:inline-flex; align-items:center; gap:8px; margin-right:10px;";

  const label = document.createElement("span");
  label.id = "figureLayerLabel";       // filled by updateLayerInfo() in figureLayers.js
  info.appendChild(label);

  const showLabel = document.createElement("label");
  showLabel.style.cssText = "display:inline-flex; align-items:center; gap:3px; margin:0;";
  showLabel.title = "Show or hide the active layer while editing. The final drawing always shows
everything.";
  const box = document.createElement("input");
  box.type = "checkbox";
  box.id   = "figureLayerShow";
  box.onchange = () => {
    const fig = uiState.figures.tabs[tabId]?.figure;
    if (fig && fig.activeLayer >= 0) setLayerVisible(tabId, fig.activeLayer, box.checked);
  };
  showLabel.appendChild(box);
  showLabel.appendChild(document.createTextNode("Show"));
  info.appendChild(showLabel);

  btnRow.insertBefore(info, btnRow.firstChild);

} // end addCaptionTools


/* ============================================================
   updateFigureButtons()
   ------------------------------------------------------------
   The left command area. Create Figure shows only on the
   Categories tab; the Insert Object list shows only on a figure
   tab. Both are created by figures.js (setFiguresAction).

   Also wires the Points Mode and Show All checkboxes to the active
   figure, grays out the insert tools in points mode, and refills
   the Insert Object list. It runs on every subtab change, so the
   wiring always points at the figure on screen.
   ============================================================ */
export function updateFigureButtons() {

  const tabId = uiState.figures.activeSubtab;
  // "!!x" turns any value into a plain true or false. live: a figure
  // subtab is showing (not Categories).
  const live  = !!uiState.figures.tabs[tabId]?.figure;

  // Small helper: display "" means the element's normal display,
  // "none" hides it.
  const show = (id, on) => {
    const el = document.getElementById(id);
    if (el) el.style.display = on ? "" : "none";
  };
  show("createFigureButton", !live);
  show("figureBr1",          !live);
```

```
  show("figureInsertSelect", live);
  show("figureBr2",          live);
  show("figurePreviewLabel",  live);
  show("figureBrPreview",     live);
  show("figureModeLabel",     live);
  show("figureBrMode",        live);

  const ts = uiState.figures.tabs[tabId];
  const pointsMode = !!ts?.mode && ts.mode === "points";

  // Points Mode checkbox. Switching mode clears both selections (a disc
  // selected in points mode means nothing in lines mode, and the
  // centroid is hidden in points mode), then rebuilds the controls and
  // listeners for the new mode (renderLayerPanel) and redraws.
  const modeCheck = document.getElementById("figureModeCheck");
  if (modeCheck) {
    modeCheck.checked = pointsMode;
    modeCheck.onchange = () => {
      const t = uiState.figures.tabs[uiState.figures.activeSubtab];
      if (!t || !t.figure) return;
      t.mode = modeCheck.checked ? "points" : "lines";
      t.selectedPoint = null;
      t.centroidSelected = false;
      updateFigureButtons();
      renderLayerPanel(uiState.figures.activeSubtab);
      redrawFigure(uiState.figures.activeSubtab);
    };
  }

  // Show All checkbox: tabState.preview. redrawFigure reads it.
  const prev = document.getElementById("figurePreviewCheck");
  if (prev) {
    prev.checked = !!uiState.figures.tabs[tabId]?.preview;
    prev.onchange = () => {
      const t = uiState.figures.tabs[uiState.figures.activeSubtab];
      if (!t || !t.figure) return;
      t.preview = prev.checked;
      redrawFigure(uiState.figures.activeSubtab);
    };
  }

  // Points mode places discs on the active layer only, so the tools that
  // add layers are grayed out and inactive until it is turned off.
  for (const id of ["figureInsert_drawLine", "figureInsert_parabola", "figureInsertSelect"]) {
    const el = document.getElementById(id);
    if (el) el.disabled = pointsMode;
  }

  const select = document.getElementById("figureInsertSelect");
  if (!select) return;

  // Refill the list: a title entry, then every registry object by name.
  select.innerHTML = "";
  const first = document.createElement("option");
  first.value = "";
  first.textContent = "Insert Object";
  select.appendChild(first);

  for (const obj of listInsertableObjects()) {
    const opt = document.createElement("option");
    opt.value = obj.key;
    opt.textContent = obj.name;
    select.appendChild(opt);
  }

```

```
  // Choosing an object inserts it and puts the list back on its title
  // entry, so the same object can be chosen again right away.
  select.onchange = () => {
    const key = select.value;
    select.value = "";
    if (key && live) insertLayer(tabId, key);
  };

} // end updateFigureButtons


/* ============================================================
   switchToCategories()
   ------------------------------------------------------------
   Shows the category list (#text) and hides the drawing area, the
   layer strip and the controls.
   ============================================================ */
export function switchToCategories() {

  setCurrentTabId("tab-categories");
  uiState.figures.activeSubtab = "tab-categories";
  renderSubtabs();

  document.getElementById("text").style.display = "block";

  const wrapper = document.getElementById("sketchpad-wrapper");
  if (wrapper) wrapper.style.display = "none";

  document.getElementById("action").style.display = "none";

  setFiguresCaption("Figures", false);

  const btnContainer = document.getElementById("figure-sidebar");
  if (btnContainer) {
    btnContainer.innerHTML  = "";
    btnContainer.style.display = "none";
  }

  // Set elsewhere when the saved-figure list has changed; reload it once.
  // import(...).then(...) loads the module and runs the function when ready.
  if (uiState.figures.needsUpdate) {
    uiState.figures.needsUpdate = false;
    import("/ui/figures/figuresDisplay.js").then(m => m.loadFiguresCategories());
  }

} // end switchToCategories


/* ============================================================
   switchToFigureTab(tabId)
   ------------------------------------------------------------
   Shows one figure: hides the category list, shows the drawing
   area and the layer strip, builds the caption bar and its tools,
   then the layer panel (which also arms the mouse handling) and
   the picture.
   ============================================================ */
export function switchToFigureTab(tabId) {

  const tabState = uiState.figures.tabs[tabId];
  if (!tabState) return;

  setCurrentTabId(tabId);
  uiState.figures.activeSubtab = tabId;
  renderSubtabs();

```

```
  document.getElementById("text").style.display = "none";

  const wrapper = document.getElementById("sketchpad-wrapper");
  if (wrapper) wrapper.style.display = "flex";

  const sidebar = document.getElementById("figure-sidebar");
  if (sidebar) sidebar.style.display = "flex";

  const action = document.getElementById("action");
  action.style.display = "block";

  setFiguresCaption(tabState.figure.name, true);
  addCaptionTools(tabId);
  updateFigureButtons();     // again: the caption tools exist only now

  // Layer list, controls of the active layer, and the picture.
  renderLayerPanel(tabId);
  redrawFigure(tabId);

} // end switchToFigureTab


/* ============================================================
   openNewFigureTab()
   ------------------------------------------------------------
   Creates an empty, unnamed figure in a new subtab and shows it.
   The figure lives in uiState.figures.tabs[tabId].figure, so it
   survives switching to another main tab and back.
   ============================================================ */
export function openNewFigureTab() {

  const tabs  = uiState.figures.tabs;
  const tabId = nextFigureTabId(tabs);

  // instances is not set here; insertLayer() creates the array with the
  // first layer. preview is not set either; undefined counts as false.
  tabs[tabId] = {
    type:        "figure",
    figure:      createEmptyFigure(nextUntitledName(tabs)),
    mode:        "lines",     // "lines" or "points" (view state, not saved)
    selectedPoint: null,
    centroidSelected: false   // the active layer's centroid is selected (view state)
  };

  switchToFigureTab(tabId);

} // end openNewFigureTab


/* ============================================================
   closeTab(tabId)
   ------------------------------------------------------------
   Throws the figure away (there is no Save yet, and no warning),
   stops every mouse listener that may belong to it, and shows
   another subtab.
   ============================================================ */
export function closeTab(tabId) {

  delete uiState.figures.tabs[tabId];
  if (window.disarmInteractor) window.disarmInteractor();
  deactivatePointsInput();
  deactivateCentroid();

  if (uiState.figures.activeSubtab === tabId) {
    switchToCategories();
```

```
  } else {
    renderSubtabs();
  }

} // end closeTab
```

figureLayers.js

ui/figures/figureLayers.js · 625 lines

INSERT, DRAW, SELECT AND HIDE LAYERS

← Contents

```
/* figureLayers.js
   ============================================================
   Figures Tab -- Layers: Insert, Draw, Select, Show/Hide
   ============================================================
   Role:
     Turns the layer data in a figure into pictures and controls.

   A layer (data, in figure.layers[i]):
     { kind: "draw", id: "<registry key>", visible: true, color: "#rrggbb",
       params: {...}, transform: { angle, tx, ty } }

   transform moves and turns the whole layer (Phase 7b, figureCentroid.js).
   The object keeps drawing in its own coordinates; drawLine maps them to the
   canvas through the transform of the layer being drawn (set here in
   redrawFigure, in drawState.transform). The same pass records the extent of
   what was drawn in instance.bounds, which gives the layer its centroid
   and bounding box.

   color is the layer's identity color: it fills the layer's number in the
   strip and the layer's discs. It is not the color of the drawing. A new
   layer takes the next color of LAYER_COLORS; right-click the number of
   a layer in the strip to pick another.

   Each layer also has a live instance, kept in
   uiState.figures.tabs[tabId].instances[i] (same index). The instance
   is Object.create(<registry entry>) with its own params and
   elements, so two layers of the same object (two parabolas) do not
   share state. The instance is not part of the figure data.

   Data and instance share one params object: layer.params and
   instance.params are the same object in memory (insertLayer sets
   layer.params = instance.params). A slider or a handle drag changes
   instance.params, and the figure data sees the change at once with
   no copying. That is the "state is written through immediately"
   rule from CLAUDE.md.

   Layer numbers are stable: layer N is figure.layers[N-1] for good.
   Stacking is separate: figure.drawOrder lists layer indices, top
   first. Selecting a layer moves it to the front of drawOrder
   (MOVE_TO_FRONT_ON_SELECT), so the last layer clicked is drawn last,
   on top. Example, layers 1,2,3 inserted: order 3 2 1; click 1 gives
   1 3 2; click 2 gives 2 1 3.

   Drawing: visible layers bottom to top, i.e. drawOrder backwards.
   Every layer except the active one is dimmed (DIM_ALPHA) while
   editing, and so are its discs (DISC_DIM_ALPHA in figurePoints.js).
   Dimming is skipped, and hidden layers are drawn, when
   drawState.final is set or the tab is in preview (Show All), or
   when redrawFigure is called with { dim: false }. Points mode obeys
   the same rules; it only changes what each layer draws (just its
   points) and what a click does.

   Controls: the active layer's controls are built with the same
   buildParameterControls() the Draw tab uses.

   What is on screen, in front-to-back order (each its own canvas,
   stacked; see canvasLayerManager.js):
     interaction  the active object's handles (interactor.js), and in
```

```
                    points mode the mouse input (figurePointer.js)
      bbox          the centroid diamond, box and rotation handle
                    (figureCentroid.js)
      main canvas   the layers, then the discs, then the weld marks
                    (redrawFigure below)

    Architectural rules:
      * Does NOT build subtabs or switch them. That is figuresNav.js.
      * Never renumbers or deletes layers (not in this phase).

    Exports:
      listInsertableObjects()
      insertLayer(tabId, registryKey, { params, transform })
      canBreakApart(tabId)
      breakApartLayer(tabId)
      redrawFigure(tabId)
      renderLayerPanel(tabId)  (layer numbers in the sidebar, heading and
                                controls in the action panel)
      selectLayer(tabId, index)
      setLayerVisible(tabId, index, visible)
    ============================================================ */

import { resetCanvas, drawState }    from "/draw/drawState.js";
import { buildParameterControls }   from "/ui/parameterControls.js";
import { syncOverlayToCanvas }      from "/ui/uiUtilities.js";
import { refreshPoints, drawFigurePoints } from "./figurePoints.js";
import { activatePointsInput, deactivatePointsInput } from "./figurePointer.js";
import { activateCentroid, centroidHit, drawCentroid } from "./figureCentroid.js";
import { snapshotWeldMembers, propagateAfterChange, drawWeldMarks } from "./figureWelds.js";
import { newTransform, newBounds } from "/draw/drawTransform.js";

// Experiment switch: true = the last layer clicked goes on top.
const MOVE_TO_FRONT_ON_SELECT = true;
// Alpha for layers that are not the active one.
const DIM_ALPHA   = 0.2;
// Identity colors handed out to layers in turn (dark enough for white text).
const LAYER_COLORS = [
  "#c0392b", "#2471a3", "#1e8449", "#d68910",
  "#7d3c98", "#117a8b", "#a04000", "#566573"
];
// Offset in canvas pixels per earlier layer of the same object.
const INSERT_STEP = 20;


/* ============================================================
   listInsertableObjects()
   ------------------------------------------------------------
   Registry objects for the Insert pulldown: [{ key, name }],
   sorted by name.

   Read as a pipeline over the registry's keys:
     filter  drop any key whose entry is missing
     map     turn each key into { key, name }
     sort    alphabetical by name (localeCompare is the
             language-aware string comparison)
   ============================================================ */
export function listInsertableObjects() {

  const reg = window.drawRegistry || {};

  return Object.keys(reg)
    .filter((key) => reg[key])
    .map((key) => ({ key, name: reg[key].name || key }))
    .sort((a, b) => a.name.localeCompare(b.name));

```

```
} // end listInsertableObjects


/* ============================================================
   insertLayer(tabId, registryKey)
   ------------------------------------------------------------
   Appends a new layer at the top of the stack with the object's
   default parameters, makes it active, and redraws.

   The optional third argument, { params, transform }, is used by
   Break Apart (a piece arrives with exact values) and by the tests
   (to place objects exactly). "= {}" at the end of the parameter list
   lets callers leave the whole third argument out.

   Steps:
     1. make a private instance of the registry object
     2. let it set itself up (init), then shift it if an identical
        object is already in the figure, so the two don't overlap
     3. record the layer data and the instance at the same index
     4. attach the callbacks that interactor.js and the weld code use
     5. put it on top, make it active, redraw
   ============================================================ */
export function insertLayer(tabId, registryKey, { params = null, transform = null } = {}) {

  const tabState = uiState.figures.tabs[tabId];
  if (!tabState || !tabState.figure) throw new Error("insertLayer: no figure for " + tabId);

  const entry = window.drawRegistry?.[registryKey];
  if (!entry) throw new Error("insertLayer: registry object not found: " + registryKey);

  // The registry entry is a shared singleton. Make a private instance.
  // Object.create(entry) makes a new, empty object that falls back on
  // entry for anything it doesn't have itself -- so it shares entry's
  // functions (init, update, draw, infrastructure ...) but gets its own
  // params and elements below. structuredClone makes a deep copy, so
  // no nested point object is shared with the registry either.
  const instance    = Object.create(entry);
  instance.params   = structuredClone(entry.params);
  instance.elements = null;
  // Break apart hands over the exact values the piece needs.
  if (params) Object.assign(instance.params, structuredClone(params));

  const figure = tabState.figure;
  // Colors cycle: the ninth layer gets the first color again ("%" is
  // the remainder after division).
  const color  = LAYER_COLORS[figure.layers.length % LAYER_COLORS.length];

  instance.init();
  // Identical objects must not land on the same spot. init() fills in
  // the default positions, so the shift comes after it.
  const same = figure.layers.filter((l) => l.id === registryKey).length;
  if (same > 0 && !params) offsetParams(instance.params, same * INSERT_STEP);

  if (!tabState.instances) tabState.instances = [];

  const layer = {
    kind:      "draw",
    id:        registryKey,
    visible:   true,
    color:     color,
    params:    instance.params,       // the same object, not a copy (see header)
    transform: transform ? { ...transform } : newTransform()   // { ...t } is a shallow copy
  };
  figure.layers.push(layer);
  tabState.instances.push(instance);
```

```
189
190   // The new layer's index. Captured here, so the callbacks below always
191   // refer to this layer, whatever happens to the stacking later.
192   const index = figure.layers.length - 1;
193   instance.redrawHandler = () => redrawFigure(tabId);
194   // Read by interactor.js: where this layer's handles really are, and
195   // whether the centroid has the press instead.
196   instance.getTransform  = () => layer.transform;
197   instance.yieldTo       = (x, y) => centroidHit(uiState.figures.tabs[tabId], x, y);
198   // Phase 8: a handle's own weld followers. beforeMove snapshots this
199   // layer's welded discs; afterMove (called once the drag has synced and
200   // redrawn the layer) compares and walks the weld graph outward.
201   // weldSnapshot is a variable private to this one layer's callbacks
202   // (a "closure"): each inserted layer has its own.
203   let weldSnapshot = null;
204   instance.beforeMove = () => {
205     weldSnapshot = snapshotWeldMembers(figure, index);
206   };
207   instance.afterMove = () => {
208     propagateAfterChange(tabId, index, weldSnapshot);
209   };
210   tabState.centroidSelected = false;
211   figure.drawOrder.unshift(index);      // new layer goes on top (unshift adds at the front)
212   figure.activeLayer = index;
213
214   renderLayerPanel(tabId);
215   redrawFigure(tabId);
216
217 } // end insertLayer
218
219
220 /* ============================================================
221    canBreakApart(tabId)
222    ------------------------------------------------------------
223    True when the active layer is visible and its object can come apart
224    (its registry entry has a decompose()).
225    ============================================================ */
226 export function canBreakApart(tabId) {
227
228   const ts = uiState.figures.tabs[tabId];
229   const f  = ts?.figure;
230   if (!f || f.activeLayer < 0) return false;
231   return !!f.layers[f.activeLayer].visible &&
232          typeof ts.instances?.[f.activeLayer]?.decompose === "function";
233
234 } // end canBreakApart
235
236
237 /* ============================================================
238    breakApartLayer(tabId)
239    ------------------------------------------------------------
240    Replaces the active layer's object with its pieces (Phase 7c). The
241    object's decompose() returns [{ key, params }]; each piece becomes a
242    new layer of its own, on top of the stack, with the same transform, so
243    the picture does not change. The original layer is hidden, not deleted:
244    layer numbers are never reused and deleting layers waits for Phase 11.
245    Its discs stay with it. The last piece becomes the active layer.
246    ============================================================ */
247 export function breakApartLayer(tabId) {
248
249   if (!canBreakApart(tabId)) return;
250
251   const ts     = uiState.figures.tabs[tabId];
252   const figure = ts.figure;
253   const index  = figure.activeLayer;
```

```
  const layer  = figure.layers[index];
  const pieces = ts.instances[index].decompose();

  layer.visible          = false;
  ts.selectedPoint       = null;
  ts.centroidSelected    = false;

  // insertLayer copies the transform ({ ...transform }), so the pieces
  // can later be moved without moving each other or the hidden original.
  for (const piece of pieces) {
    insertLayer(tabId, piece.key, { params: piece.params, transform: layer.transform });
  }

} // end breakApartLayer


/* ------------------------------------------------------------
   offsetParams(params, d)
   Shifts whatever positions a registry object keeps: its points,
   midpoint, center or start. Objects with none of these are
   not moved.

   Registry objects are not uniform about where they keep their
   positions, so this tries the four common shapes. It moves only
   the handles array (params.points) for a Line or a Parabola; the
   object's own update() then reads the moved handles back into its
   named fields (pt1, line1_pt1 ...) on the next draw.
   ------------------------------------------------------------ */
function offsetParams(params, d) {

  if (Array.isArray(params.points)) {
    for (const pt of params.points) {
      if (typeof pt.x === "number") { pt.x += d; pt.y += d; }
    }
  }
  if (params.midpoint && typeof params.midpoint.x === "number") {
    params.midpoint.x += d;
    params.midpoint.y += d;
  }
  // "kx in params" asks whether params has a field named kx at all.
  for (const [kx, ky] of [["centerX", "centerY"], ["startX", "startY"]]) {
    if (kx in params && ky in params) {
      params[kx] += d;
      params[ky] += d;
    }
  }

} // end offsetParams


/* ============================================================
   redrawFigure(tabId)
   ------------------------------------------------------------
   Clears the canvas and draws every visible layer, bottom to top.
   A layer that throws is reported with its index and skipped.

   This is the one place a figure is drawn. Nearly every change ends
   by calling it: a slider, a handle drag (through
   instance.redrawHandler), a disc click, a weld, a mode switch. It
   draws in three passes:
     1. the layers (each object's own draw(), through drawLine)
     2. the discs (figurePoints.js), after recomputing where anchored
        ones now are
     3. the weld marks (figureWelds.js), then the centroid overlay
   ============================================================ */
```

```
export function redrawFigure(tabId, { dim = true } = {}) {

  const tabState = uiState.figures.tabs[tabId];
  if (!tabState || !tabState.figure) return;

  resetCanvas();

  const figure    = tabState.figure;
  const instances = tabState.instances || [];
  // Preview (the Show All checkbox, not saved) and final mode draw every
  // layer at full strength, hidden ones included. Layers of kind "template"
  // (guides, a later phase) are left out of both. Points mode sets
  // drawState.final while the layers draw, so they show only their points
  // (one flag, set here, restored below), but it follows the same
  // visibility and dimming rules as lines mode.
  const pointsMode = tabState.mode === "points";
  const showAll    = drawState.final || !!tabState.preview;
  const dimming    = dim && !showAll;
  const wasFinal   = drawState.final;
  if (pointsMode) drawState.final = true;

  // drawOrder is top first; paint bottom to top.
  for (let k = figure.drawOrder.length - 1; k >= 0; k--) {
    const i = figure.drawOrder[k];
    if (showAll && figure.layers[i].kind === "template") continue;
    if (!figure.layers[i].visible && !showAll) continue;

    const inst = instances[i];
    // save()/restore() bracket each layer so its alpha (dimming) and any
    // style it sets do not leak into the next layer.
    ctx.save();
    if (dimming && i !== figure.activeLayer) ctx.globalAlpha = DIM_ALPHA;
    // drawLine maps what the object draws through this layer's transform and
    // notes its extent in drawState.bounds. Both are null again afterwards.
    drawState.transform = figure.layers[i].transform || null;
    drawState.bounds    = newBounds();
    // update() first: it brings the object's internal geometry in line
    // with its params (a slider or a handle may have changed them).
    try {
      inst.update(inst.params);
      inst.draw();
    } catch (err) {
      console.error(`Figure layer ${i + 1} (${figure.layers[i].id}) failed to draw:`, err);
    }
    inst.bounds         = drawState.bounds;    // kept for figureCentroid.js
    drawState.transform = null;
    drawState.bounds    = null;
    ctx.restore();
  }

  drawState.final = wasFinal;

  // Discs are always shown, in both modes, by the same visibility and dimming
  // rules as the layers. Anchored ones follow their layers, so refresh first.
  refreshPoints(tabState);
  drawFigurePoints(tabState, { showAll, dimming });
  drawWeldMarks(tabState);

  // The active layer's centroid, and its box and rotation handle when selected.
  drawCentroid(tabState);

} // end redrawFigure


/* ============================================================
```

```
    renderLayerPanel(tabId)
    ------------------------------------------------------------
    Layer numbers go in the gray strip
    left of the canvas (#figure-sidebar); a hidden layer's number is
    grayed and struck through. The action panel gets the active
    layer's controls. The subtab bar item for the active layer
    (name and Show checkbox) is refreshed. Arms the point
    interactor (the object's handles) on the active layer when it is
    visible, or always in points mode, and the points-mode mouse
    input as well in points mode.

    Called whenever which layer is active, or the mode, changes. It
    tears down every mouse listener (handles, points mode, centroid)
    and sets up the ones that fit the new situation, so nothing from
    the previous layer is ever left listening.
    ============================================================ */
export function renderLayerPanel(tabId) {

  const tabState = uiState.figures.tabs[tabId];
  const action   = document.getElementById("action");
  const strip    = document.getElementById("figure-sidebar");
  if (!tabState || !tabState.figure || !action) return;

  const figure    = tabState.figure;
  const instances = tabState.instances || [];

  // ---- layer numbers in the strip ---------------------------------
  if (strip) {
    strip.innerHTML = "";

    figure.layers.forEach((layer, i) => {

      const reg = window.drawRegistry?.[layer.id];

      const row = document.createElement("div");
      row.className = "figure-layer-row";
      row.title = reg?.name || layer.id;      // the tooltip names the object
      row.style.cssText =
        "display:flex; align-items:center; justify-content:center; width:100%; padding:1px 4px;
cursor:pointer;" +
        (i === figure.activeLayer ? " background:rgba(0,0,0,0.18); font-weight:bold;" : "");

      // The number sits on a chip in the layer's color, the same color as the
      // layer's discs. Right-click it to choose another color.
      if (!layer.color) layer.color = LAYER_COLORS[i % LAYER_COLORS.length];

      const num = document.createElement("span");
      num.textContent = String(i + 1);
      num.style.cssText =
        "display:inline-block; min-width:22px; padding:1px 0; border-radius:11px; text-align:center;"
+
        " background:" + layer.color + "; color:" + textColorOn(layer.color) + ";";
      num.title = "Right-click to change this layer's color";

      // Hidden native color picker, opened by the right-click.
      const picker = document.createElement("input");
      picker.type  = "color";
      picker.value = layer.color;
      picker.style.cssText = "position:absolute; width:0; height:0; opacity:0; pointer-
events:none;";
      picker.oninput = () => {
        layer.color = picker.value;
        num.style.background = layer.color;
        num.style.color      = textColorOn(layer.color);
        redrawFigure(tabId);
```

```
      };
      num.oncontextmenu = (e) => {
        e.preventDefault();       // no browser menu
        picker.click();           // opens the system color chooser
      };

      if (!layer.visible) {
        row.style.opacity = "0.4";
        num.style.textDecoration = "line-through";
        row.title += " (hidden)";
      }

      row.onclick = () => selectLayer(tabId, i);
      row.appendChild(num);
      row.appendChild(picker);
      strip.appendChild(row);
    });
  }

  // ---- heading and controls for the active layer ---------------------
  action.innerHTML = "";
  const active = instances[figure.activeLayer];

  if (!active) {
    action.textContent = "Use Insert Object, Line or Parabola to begin.";
  } else {
    const info = {
      drawRegistry:  active,          // controls come from the registry entry
      parameters:    active.params,
      redrawHandler: () => redrawFigure(tabId),
      onParamChange: () => {}         // dirty tracking arrives with Save As
    };
    buildParameterControls(info, "tab-draw", true);
  }

  updateLayerInfo(tabId);

  // ---- interactor (drag handles) -------------------------------------
  // The overlay canvases must be the same size as the main canvas, or a
  // handle drawn on them would not sit over the point it belongs to.
  const canvas = window.drawCanvas;
  if (canvas) {
    syncOverlayToCanvas("interaction", canvas);
    syncOverlayToCanvas("guides", canvas);
    syncOverlayToCanvas("bbox", canvas);
  }

  // The active object's own handles are the only draggable points, in both
  // modes, and a hidden layer has none. In points mode figurePointer.js also
  // takes clicks (place, select, delete). It is set up after the interactor:
  // an object with no handles makes the interactor hide the interaction
  // layer, and the pointer input has to show it again.
  if (window.disarmInteractor) window.disarmInteractor();
  deactivatePointsInput();

  const pointsMode = tabState.mode === "points";
  if (active && figure.layers[figure.activeLayer].visible) {
    if (window.armInteractor) window.armInteractor(active);
  }
  if (pointsMode) activatePointsInput(tabId, () => redrawFigure(tabId));
  activateCentroid(tabId, () => redrawFigure(tabId));

} // end renderLayerPanel


```

```
/* ------------------------------------------------------------
   textColorOn(hex)
   Black or white, whichever reads better on the given background.

   "#c0392b" -> parseInt("c0392b", 16) is one number holding all three
   color bytes; ">> 16" and "& 255" pick out red, ">> 8" green, the
   last byte blue. The weights 0.299, 0.587, 0.114 are the standard
   perceived-brightness formula (green looks brightest to the eye).
   ------------------------------------------------------------ */
function textColorOn(hex) {

  const n = parseInt(hex.slice(1), 16);
  const r = (n >> 16) & 255;
  const g = (n >> 8)  & 255;
  const b = n & 255;
  return (0.299 * r + 0.587 * g + 0.114 * b) > 150 ? "#000000" : "#ffffff";

} // end textColorOn


/* ============================================================
   updateLayerInfo(tabId)
   ------------------------------------------------------------
   Fills the "Layer N . Name  [x] Show" item in the subtab bar
   (built by figuresNav.js) for the active layer.
   ============================================================ */
function updateLayerInfo(tabId) {

  const label = document.getElementById("figureLayerLabel");
  const box   = document.getElementById("figureLayerShow");
  if (!label || !box) return;

  const figure = uiState.figures.tabs[tabId].figure;
  const layer  = figure.layers[figure.activeLayer];

  if (!layer) {
    label.textContent = "";
    box.checked  = true;
    box.disabled = true;
    return;
  }

  const reg = window.drawRegistry?.[layer.id];
  // · is the middle dot character.
  label.textContent = "Layer " + (figure.activeLayer + 1) + " · " + (reg?.name || layer.id);
  box.checked  = layer.visible;
  box.disabled = false;

} // end updateLayerInfo


/* ============================================================
   selectLayer(tabId, index)
   ------------------------------------------------------------
   Makes a layer active (a click on its number in the strip).
   ============================================================ */
export function selectLayer(tabId, index) {

  const tabState = uiState.figures.tabs[tabId];
  if (!tabState) return;

  const figure = tabState.figure;
  figure.activeLayer = index;
  tabState.selectedPoint = null;   // only the active layer's points can be selected
  tabState.centroidSelected = false;
```

```
576
577   // [index, ...rest] builds a new array: index first, then every other
578   // entry of drawOrder in its old order.
579   if (MOVE_TO_FRONT_ON_SELECT) {
580     figure.drawOrder = [index, ...figure.drawOrder.filter((n) => n !== index)];
581   }
582
583   renderLayerPanel(tabId);
584   redrawFigure(tabId);
585
586 } // end selectLayer
587
588
589 /* ============================================================
590    setLayerVisible(tabId, index, visible)
591    ------------------------------------------------------------
592    The Show checkbox in the caption bar. Hiding is for editing only:
593    Show All and the final drawing still include hidden layers.
594    ============================================================ */
595 export function setLayerVisible(tabId, index, visible) {
596
597   const tabState = uiState.figures.tabs[tabId];
598   if (!tabState) return;
599
600   const figure = tabState.figure;
601   figure.layers[index].visible = !!visible;
602   tabState.centroidSelected = false;
603
604   // A hidden layer has nothing to edit, so hiding the active layer makes the
605   // next visible layer down the stack active (or, if none is below, any
606   // visible layer). The stacking itself is left alone (unlike selectLayer,
607   // which moves the layer to the front).
608   // "a ?? b" gives b only when a is null or undefined (unlike "||", which
609   // would also skip a layer index of 0).
610   if (!visible && index === figure.activeLayer) {
611     const order = figure.drawOrder;
612     const isVisible = (i) => figure.layers[i].visible;
613     const next  = order.slice(order.indexOf(index) + 1).find(isVisible) ?? order.find(isVisible);
614     if (next !== undefined) {
615       figure.activeLayer = next;
616       tabState.selectedPoint = null;
617     }
618   }
619
620   renderLayerPanel(tabId);
621   redrawFigure(tabId);
622
623 } // end setLayerVisible
624
625 // end figureLayers.js
```

drawTransform.js

MOVING AND TURNING: THE MATH

← Contents

```
/* draw/drawTransform.js
   ============================================================
   The layer transform: object space to canvas space
   ============================================================
   Role:
     In a figure every layer can be moved and rotated without touching the
     object's own numbers. An object keeps drawing in "object space" (its
     own coordinates); the layer's transform turns those into canvas
     coordinates. No canvas operations (ctx.translate, ctx.rotate) are used.

   A transform is a plain object:  { angle, tx, ty }
     angle  rotation in radians
     tx, ty offset in canvas pixels
   canvas point = rotate(object point, angle) + (tx, ty)
   The identity transform is { angle: 0, tx: 0, ty: 0 }. A missing (null or
   undefined) transform means the identity, so the Draw tab never has to
   know about any of this.

   Two pieces of drawState are involved (declared in drawState.js):
     drawState.transform  the transform of the layer being drawn, or null.
                          drawLine and mapPoint apply it.
     drawState.bounds     { minX, minY, maxX, maxY } of everything drawn in
                          object space, or null. drawLine fills it in so a
                          layer's centroid and bounding box can be found.
   Figure code sets both before a layer draws and clears them after.

   Why object space at all:
     A Line's handles say pt1 = (200, 200). If the layer is moved 100 px
     right, the handle should still say (200, 200), so the object's own
     sliders, steps and handle logic never need to know it was moved.
     Only the layer's transform changes, to tx = 100. Everything that
     shows, hits or drags a handle or disc goes through the same transform
     (interactor.js through getTransform, figurePoints.js through
     armPoint), so what you see and what you grab always agree.

   A worked example, a quarter turn and a shift:
     t = { angle: Math.PI / 2, tx: 300, ty: 100 }     (90 degrees)
     object point (10, 0):
       rotate 90 degrees  -> (0, 10)      cos = 0, sin = 1:
                                          x' = 0*10 - 1*0 = 0
                                          y' = 1*10 + 0*0 = 10
       then add (tx, ty)  -> (300, 110)
     canvas y grows downward, so a positive angle turns clockwise on
     screen.

   Exports:
     newTransform()                       the identity transform
     applyTransform(t, pt)                object point to canvas point
     inverseTransform(t, x, y)            canvas point to object point
     translateTransform(t, dx, dy)        a copy moved by (dx, dy) canvas px
     rotateTransformAbout(t, cx, cy, d)   a copy turned by d radians about
                                          the canvas point (cx, cy)
     mapPoint(pt)                         applyTransform with drawState.transform
     newBounds()                          an empty bounds object
     noteBounds(pt)                       grow drawState.bounds to include pt
   ============================================================ */

import { drawState } from "./drawState.js";
```

```
59
60
61  // A fresh identity transform: no turn, no shift. A new object each call,
62  // so two layers never share (and so never jointly change) one transform.
63  export function newTransform() {
64    return { angle: 0, tx: 0, ty: 0 };
65  } // end newTransform
66
67
68  /* ------------------------------------------------------------
69     applyTransform(t, pt) -> { x, y }
70     Object space to canvas space: turn about the object-space origin
71     (0, 0) by t.angle, then shift by (t.tx, t.ty). This is the standard
72     2-D rotation:
73         x' = cos * x - sin * y
74         y' = sin * x + cos * y
75     Always returns a new object; pt is never changed.
76     ------------------------------------------------------------ */
77  export function applyTransform(t, pt) {
78
79    if (!t) return { x: pt.x, y: pt.y };     // no transform: the identity
80
81    const c = Math.cos(t.angle);
82    const s = Math.sin(t.angle);
83    return {
84      x: c * pt.x - s * pt.y + t.tx,
85      y: s * pt.x + c * pt.y + t.ty
86    };
87
88  } // end applyTransform
89
90
91  /* ------------------------------------------------------------
92     inverseTransform(t, x, y) -> { x, y }
93     The object point that the transform sends to canvas point (x, y).
94     Undoes applyTransform in reverse order: first take the shift off,
95     then turn back by -angle (cos(-a) = cos a, sin(-a) = -sin a, which
96     is where the signs below come from). Used whenever a mouse position
97     (canvas space) has to become an object's own coordinates: dragging
98     a handle, placing or dragging a free disc.
99     ------------------------------------------------------------ */
100 export function inverseTransform(t, x, y) {
101
102   if (!t) return { x, y };
103
104   const c  = Math.cos(t.angle);
105   const s  = Math.sin(t.angle);
106   const ux = x - t.tx;          // shift taken off
107   const uy = y - t.ty;
108   return {
109     x:  c * ux + s * uy,        // turned back by -angle
110     y: -s * ux + c * uy
111   };
112
113 } // end inverseTransform
114
115
116 /* ------------------------------------------------------------
117    translateTransform(t, dx, dy) -> a new transform
118    The same transform, moved (dx, dy) canvas pixels. The angle is
119    untouched: moving never turns. t may be null (the identity).
120    ------------------------------------------------------------ */
121 export function translateTransform(t, dx, dy) {
122
123   const from = t || newTransform();
```

```
  return { angle: from.angle, tx: from.tx + dx, ty: from.ty + dy };

} // end translateTransform


/* ------------------------------------------------------------
   rotateTransformAbout(t, cx, cy, d) -> a new transform
   Turns the whole picture by d radians about the canvas point (cx, cy).
   A canvas point p goes to  c + R(d) * (p - c); applying that to
   R(angle) * object + (tx, ty) gives the new angle and offset below.

   In words: the angles add (angle + d), and the old shift (tx, ty),
   measured from the pivot (rx, ry), is itself turned by d about the
   pivot. The figure's centroid drag (figureCentroid.js) uses this with
   the pivot at the layer's centroid, so the layer turns in place.
   ------------------------------------------------------------ */
export function rotateTransformAbout(t, cx, cy, d) {

  const from = t || newTransform();
  const c = Math.cos(d);
  const s = Math.sin(d);
  const rx = from.tx - cx;      // the shift, seen from the pivot
  const ry = from.ty - cy;
  return {
    angle: from.angle + d,
    tx: cx + c * rx - s * ry,   // turned by d, back from the pivot
    ty: cy + s * rx + c * ry
  };

} // end rotateTransformAbout


/* ------------------------------------------------------------
   mapPoint(pt): the point as drawn, using the transform of the layer
   being drawn (drawState.transform).
   Outside a figure layer drawState.transform is null, so this returns
   the point unchanged and the Draw tab behaves as it always did.
   ------------------------------------------------------------ */
export function mapPoint(pt) {
  return applyTransform(drawState.transform, pt);
} // end mapPoint


// An empty extent: every real point will be smaller than Infinity and
// larger than -Infinity, so the first noteBounds() call sets all four.
// After drawing, minX still being Infinity means nothing was drawn.
export function newBounds() {
  return { minX: Infinity, minY: Infinity, maxX: -Infinity, maxY: -Infinity };
} // end newBounds


/* ------------------------------------------------------------
   noteBounds(pt): include pt in drawState.bounds (object space), if the
   caller has asked for bounds by setting drawState.bounds.
   drawLine calls this for both ends of every line it draws. Object
   space, not canvas space, so the box can be turned with the layer
   (figureCentroid.js) instead of growing as the layer turns.
   ------------------------------------------------------------ */
export function noteBounds(pt) {

  const b = drawState.bounds;
  if (!b) return;               // nobody asked: the Draw tab, or no figure

  if (pt.x < b.minX) b.minX = pt.x;
  if (pt.y < b.minY) b.minY = pt.y;
```

```
  if (pt.x > b.maxX) b.maxX = pt.x;
  if (pt.y > b.maxY) b.maxY = pt.y;

} // end noteBounds

// end drawTransform.js
```

figurePoints.js

ui/figures/figurePoints.js · 388 lines

DISCS: DATA, FINDING, DRAWING

← Contents

```
/* figurePoints.js
   ============================================================
   Figures Tab -- Numbered Points (data, picking, drawing)
   ============================================================
   Role:
     The numbered points of a figure: creating, finding, moving,
     deleting and drawing them. No event handling (that is
     figurePointer.js) and no layer drawing (figureLayers.js).

   Point record, in figure.points:
     { n, kind: "anchored", layer, arm, t, x, y, dx, dy }
     { n, kind: "free",     layer,          x, y, dx, dy, ox, oy }

     n        permanent number, taken from figure.nextPoint. Never
              reused, even after the point is deleted. It is an id only:
              the disc is drawn without it.
     layer    index into figure.layers (layer numbers never change). A
              free point belongs to the layer that was active when it
              was placed. Only the active layer's points can be selected,
              added or deleted.
     arm, t   an anchored point sits on an infrastructure line of its
              layer (arm id) at fraction t of its length. It always
              keeps exactly that fraction, so it follows its object
              when the object is moved or its step count changes.
     x, y     where the point is now, in canvas pixels. For an anchored
              point this is recomputed from layer, arm and t on every
              redraw. Points are never dragged except free ones
              (moveFreePoint): an anchored point rides along with its
              object. The object's own handles are dragged by interactor.js.
     ox, oy   a free point only: where it is in its layer's object space.
              x, y are recomputed from these on every redraw, so a free
              disc travels with its layer when the layer is moved or turned
              (Phase 7b). Where the layer has no transform, ox = x, oy = y.

   Layer transform (Phase 7b): a layer may carry layer.transform (see
   draw/drawTransform.js). Infrastructure arms come from the object in object
   space; every arm is mapped through the transform of its layer before a
   disc is found or placed on it, so all disc positions are in canvas space.
     dx, dy   offset of a number label from the point. Discs are drawn
              without numbers now, so this is always 0; kept for
              Phase 7, which may drop it.

   Vocabulary: Barry's name for a drawn point is "disc" (a filled circle,
   in the color of its layer, drawn without its number). Code and tests
   still say "point".

   Infrastructure: a registry object may define infrastructure(),
   returning arms. An arm is either { id, a, b, steps, from, to } (a
   straight run between two points) or { id, points, from, to } (an
   explicit array, for an arc or a ring of nodes; steps = points.length - 1
   so the same t = i/steps still picks out index i). The candidate points
   (the places a click can pick) are t = i / steps for i = from..to.
   Stitch lines are not infrastructure and cannot carry points.

   A worked example: a Parabola with Steps 20 has two arms. Its arm2 runs
   from line2_pt1 (50, 350) to line2_pt2 (350, 350), steps 20, from 0 to
   20. The candidates are 21 spots, 15 px apart: t = 0, 0.05, 0.10 ...
   1.0. A click at (262, 351) with hit radius 5 is nearest to i = 14
```

```
   (t = 0.7, x = 50 + 300 * 0.7 = 260), 2.2 px away, so it numbers an
   anchored disc { arm: "arm2", t: 0.7 }. Drag line2_pt2 elsewhere and
   that disc stays at 70% of the way along the arm.

   Phase 8: refreshPoints() removes an anchored point outright, instead
   of keeping its last position, once its arm is gone or its t has
   fallen outside from..to (the object was replaced, or Trim Bottom /
   Shorten Top trimmed its spot away). A point that disappears this way
   leaves its weld, the same as deletePoint().

   Exports:
     armsOf(instance)
     armPoint(arm, t, transform)
     findCandidate(tabState, x, y, radius)
     findPoint(figure, x, y, radius)
     addAnchoredPoint(figure, candidate)
     addFreePoint(figure, x, y, transform)
     moveFreePoint(point, x, y, transform)
     deletePoint(figure, n)
     refreshPoints(tabState)
     drawFigurePoints(tabState, { showAll, dimming })
   ============================================================ */

import { applyTransform, inverseTransform } from "/draw/drawTransform.js";
import { removeFromWeld } from "./figureWelds.js";

const MARKER_RADIUS = 9;   // drawn size of a numbered point
const HALO_GAP      = 4;   // selected point: gap from the marker to its outer ring
// Alpha for the discs of layers that are not the active one, while editing.
// Layers themselves are dimmed harder (DIM_ALPHA in figureLayers.js).
const DISC_DIM_ALPHA = 0.5;


/* ------------------------------------------------------------
   armsOf(instance) -- infrastructure of one layer ([] if none)
   Exported for figureWelds.js (Phase 8), which has to look across an
   object's own other arms -- a Parabola's two corner handles, say --
   when a weld is made at a spot more than one arm touches.
   An object without infrastructure() (most gallery-derived objects
   before Phase 7d) simply has no arms, so no anchored discs: a click
   on it makes a free disc.
   ------------------------------------------------------------ */
export function armsOf(instance) {
  return (instance && typeof instance.infrastructure === "function")
    ? instance.infrastructure()
    : [];
} // end armsOf


// The point at fraction t along an arm, in canvas space. transform is the
// layer's transform, or null/undefined for none. Two arm shapes (Phase 7d):
// a straight run between a and b (the original shape), or an explicit
// points array for an arc or a ring of nodes -- t = i/steps still picks
// index i, since steps = points.length - 1 for that shape. Exported for
// figureWelds.js, same reason as armsOf() above.
//
// The straight case is ordinary linear interpolation: t = 0 is a, t = 1
// is b, t = 0.5 is halfway. Math.round in the array case guards against
// t * steps coming out as 6.999999 instead of 7.
export function armPoint(arm, t, transform) {
  const pt = arm.points
    ? arm.points[Math.round(t * arm.steps)]
    : {
        x: arm.a.x + (arm.b.x - arm.a.x) * t,
        y: arm.a.y + (arm.b.y - arm.a.y) * t
```

```
      };
  return applyTransform(transform, pt);
} // end armPoint


// The transform of layer number i of the figure, or null.
function layerTransform(figure, i) {
  return figure.layers[i]?.transform || null;
} // end layerTransform


/* ============================================================
   findCandidate(tabState, x, y, radius)
   ------------------------------------------------------------
   The infrastructure point nearest to (x, y) within radius, or
   null. Only the active layer is searched.
   Returns { layer, arm, t, x, y }.

   Ties: the comparison is "d < bestDist" (strictly less), so when two
   arms have a candidate at the same distance -- a Parabola's corner,
   where arm1 ends and arm2 starts -- the arm checked first (arm1)
   wins. figureWelds.js (cornerPartners) exists partly because of this.
   ============================================================ */
export function findCandidate(tabState, x, y, radius) {

  const figure    = tabState.figure;
  const instances = tabState.instances || [];
  const layer     = figure.activeLayer;
  const xform     = layerTransform(figure, layer);

  let best     = null;
  let bestDist = radius;      // anything farther than radius never qualifies

  for (const arm of armsOf(instances[layer])) {
    for (let i = arm.from; i <= arm.to; i++) {
      const t  = i / arm.steps;
      const pt = armPoint(arm, t, xform);
      const d  = Math.hypot(pt.x - x, pt.y - y);   // straight-line distance
      if (d < bestDist) {
        bestDist = d;
        best = { layer, arm: arm.id, t, x: pt.x, y: pt.y };
      }
    }
  }

  return best;

} // end findCandidate


/* ============================================================
   findPoint(figure, x, y, radius)
   ------------------------------------------------------------
   The numbered point of the active layer nearest to (x, y) within
   radius, or null. The radius is never smaller than the drawn
   marker, so a click anywhere on a marker finds it. Points of other
   layers are ignored.
   ============================================================ */
export function findPoint(figure, x, y, radius) {

  const reach = Math.max(radius, MARKER_RADIUS);

  let best     = null;
  let bestDist = reach;

```

```
  for (const p of figure.points) {
    if (p.layer !== figure.activeLayer) continue;
    const d = Math.hypot(p.x - x, p.y - y);
    if (d < bestDist) {
      bestDist = d;
      best = p;
    }
  }

  return best;

} // end findPoint


/* ============================================================
   addAnchoredPoint(figure, candidate) / addFreePoint(figure, x, y)
   ------------------------------------------------------------
   Allocate the next number and add the point. Returns the point. A
   free point goes on the active layer.

   "figure.nextPoint++" uses the current value and then adds one, so
   the first point is 1, the next 2, and so on, never repeating.
   ============================================================ */
export function addAnchoredPoint(figure, c) {

  const p = {
    n: figure.nextPoint++, kind: "anchored",
    layer: c.layer, arm: c.arm, t: c.t,
    x: c.x, y: c.y, dx: 0, dy: 0
  };
  figure.points.push(p);
  return p;

} // end addAnchoredPoint


// transform: the active layer's transform, so the free disc's position
// can be stored in that layer's object space (ox, oy) and travel with
// the layer from then on. "= null" is a default for callers that omit it.
export function addFreePoint(figure, x, y, transform = null) {

  const o = inverseTransform(transform, x, y);
  const p = {
    n: figure.nextPoint++, kind: "free", layer: figure.activeLayer,
    x, y, dx: 0, dy: 0, ox: o.x, oy: o.y
  };
  figure.points.push(p);
  return p;

} // end addFreePoint


/* ============================================================
   moveFreePoint(point, x, y)
   ------------------------------------------------------------
   Drags a free point to (x, y). An anchored point is never moved
   this way (it rides along with its object); returns false for it.

   Used by a points-mode drag (figurePointer.js) and by a weld that
   has to bring a free member along (figureWelds.js).
   ============================================================ */
export function moveFreePoint(point, x, y, transform = null) {

  if (point.kind !== "free") return false;

```

```
  const o = inverseTransform(transform, x, y);
  point.x  = x;
  point.y  = y;
  point.ox = o.x;
  point.oy = o.y;
  return true;

} // end moveFreePoint


/* ============================================================
   deletePoint(figure, n)
   ------------------------------------------------------------
   Removes the point. Its number stays used: nextPoint never
   goes back. A welded point leaves its weld first (Phase 8), the same
   as right-clicking it to unweld.

   filter() builds a new array of every point except number n, and
   that new array replaces figure.points.
   ============================================================ */
export function deletePoint(figure, n) {

  removeFromWeld(figure, n);
  figure.points = figure.points.filter((p) => p.n !== n);

} // end deletePoint


/* ============================================================
   refreshPoints(tabState)
   ------------------------------------------------------------
   Recomputes where each anchored point is from its layer's
   current geometry, keeping its fraction t. Call after the layers
   have been updated and before drawing.

   Phase 8: an anchored point whose arm has gone (object replaced) or
   whose index has fallen outside the arm's current from..to (Trim
   Bottom/Shorten Top trimmed it away) disappears with it, leaving its
   weld the same way deletePoint does -- it no longer has a real place
   to be, so there is nothing to keep showing.

   Points to remove are collected in "gone" first and removed after
   the loop: removing entries from an array while walking through it
   would skip some.
   ============================================================ */
export function refreshPoints(tabState) {

  const instances = tabState.instances || [];
  const figure    = tabState.figure;
  const gone      = [];

  for (const p of figure.points) {
    const xform = layerTransform(figure, p.layer);

    // A free disc: object-space position, through its layer's transform.
    if (p.kind === "free") {
      if (typeof p.ox !== "number") continue;
      const pt = applyTransform(xform, { x: p.ox, y: p.oy });
      p.x = pt.x;
      p.y = pt.y;
      continue;
    }

    // An anchored disc: its arm, as the object is shaped right now.
    const arm = armsOf(instances[p.layer]).find((a) => a.id === p.arm);
```

```
    if (!arm) { gone.push(p.n); continue; }

    const i = Math.round(p.t * arm.steps);
    if (i < arm.from || i > arm.to) { gone.push(p.n); continue; }

    const pt = armPoint(arm, p.t, xform);
    p.x = pt.x;
    p.y = pt.y;
  }

  for (const n of gone) {
    removeFromWeld(figure, n);
    figure.points = figure.points.filter((p) => p.n !== n);
  }

} // end refreshPoints


/* ============================================================
   drawFigurePoints(tabState, { showAll, dimming })
   ------------------------------------------------------------
   Draws every disc on the main canvas: a filled circle in the color
   of its layer with a black rim, no number. The same visibility
   rules as the layers apply: a disc of a hidden layer is left out
   unless showAll. While dimming, the discs of layers other than the
   active one are drawn at DISC_DIM_ALPHA. The selected disc has a
   thick red rim and a second red ring just outside it, so it stands
   out even when it sits on a handle or a stitch line.

   Drawing a circle on a canvas: beginPath() starts a shape,
   arc(x, y, r, start, end) traces it (0 to 2 pi radians is a full
   circle), fill() paints the inside, stroke() the outline.
   ============================================================ */
export function drawFigurePoints(tabState, { showAll = false, dimming = false } = {}) {

  const figure = tabState.figure;

  ctx.save();

  for (const p of figure.points) {
    const layer = figure.layers[p.layer];
    if (!layer) continue;
    if (!layer.visible && !showAll) continue;

    const selected = p.n === tabState.selectedPoint;

    ctx.globalAlpha = (dimming && p.layer !== figure.activeLayer) ? DISC_DIM_ALPHA : 1;

    ctx.beginPath();
    ctx.arc(p.x, p.y, MARKER_RADIUS, 0, Math.PI * 2);
    ctx.fillStyle   = layer.color || "#888888";
    ctx.fill();
    ctx.lineWidth   = selected ? 3 : 1.5;
    ctx.strokeStyle = selected ? "#d00000" : "#000000";
    ctx.stroke();

    if (selected) {
      ctx.beginPath();
      ctx.arc(p.x, p.y, MARKER_RADIUS + HALO_GAP, 0, Math.PI * 2);
      ctx.lineWidth   = 2;
      ctx.strokeStyle = "#d00000";
      ctx.stroke();
    }
  }

```

```
  ctx.restore();

} // end drawFigurePoints

// end figurePoints.js
```

figurePointer.js

ui/figures/figurePointer.js · 354 lines

THE MOUSE AND KEYS IN POINTS MODE

← Contents

```
/* figurePointer.js
   ============================================================
   Figures Tab -- Mouse and Keyboard in Points Mode
   ============================================================
   Role:
     While a figure is in points mode, the (transparent) interaction
     layer over the canvas receives the mouse. Only the active layer
     can be changed.

     Drag (press, move, release):
       on a free disc              moves it. This is the one thing in
                                   Figures that drags a disc: a free
                                   disc depends on nothing, so no
                                   drawing routine has to know about it
       on an anchored disc         nothing (it rides along with its
                                   object)
       on a handle of the object   the interactor drags the handle
                                   (interactor.js); a free disc under
                                   the handle stays put

     Click (a press that is not a drag; see below):

       on a numbered point         select it
       near a candidate            number that infrastructure point
                                   (anchored point) and select it
       anywhere else               a free point, numbered where you
                                   clicked
       Delete / Backspace          delete the selected point

     Right-click (Phase 8, figureWelds.js): toggles whether the disc
     under the cursor is welded. Already welded -> drops it out of its
     group. Not welded, with another disc (any layer, the same layer
     included, for tying a corner) at the same spot -> welds them. A
     bare infrastructure point is numbered first, but only if that
     lands it on an existing disc -- otherwise nothing happens. Either
     way a short message shows in the caption bar. The browser's own
     context menu is suppressed on both the right mousedown and the
     contextmenu event, so it never flashes up first.

     Only the left button clicks, places, selects or drags. A right
     press used to be taken as a left one too: it placed or selected a
     disc on release, and (in interactor.js) grabbed a handle, which
     any hand jitter then dragged under the cursor. That put a stray
     disc on the wrong layer and produced a "corner tied" weld of a
     line to itself (bug found in real use, 2026-10-27, PHASES.md).

     Both this file and the interactor listen at the same time in
     points mode. A press on a handle belongs to the interactor, so a
     disc is dragged only when the press is clear of the handles. A
     click on a handle does not move it, so a disc can be placed on
     an end point.

     What counts as a click: a press on a handle is a click only if it
     does not move (more than CLICK_SLOP), because a moving press is the
     handle being dragged. A press anywhere else is always a click at the
     spot where it went down, even if the hand slips, since dragging on
     empty canvas means nothing else. A press that dragged a free disc
     only selects it.
```

```
 59
 60      A hidden active layer cannot be edited: its discs would not be
 61      drawn. Clicks are ignored then and the pointer shows "not-allowed".
 62
 63      "Near" means within the hit radius, uiState.figures.hitRadius
 64      canvas pixels. Numbers are never reused.
 65
 66    Exports:
 67      activatePointsInput(tabId, redraw)
 68      deactivatePointsInput()
 69    ============================================================ */
 70
 71  import { canvasLayerManager } from "/ui/canvasLayerManager.js";
 72  import { uiState }            from "/ui/uiState.js";
 73  import { PointPickerPresets } from "/ui/interactor.js";
 74  import { applyTransform }      from "/draw/drawTransform.js";
 75  import {
 76    findCandidate,
 77    findPoint,
 78    addAnchoredPoint,
 79    addFreePoint,
 80    moveFreePoint,
 81    deletePoint
 82  } from "./figurePoints.js";
 83  import { toggleWeldAt } from "./figureWelds.js";
 84
 85  const CLICK_SLOP = 4;    // canvas pixels a press may move and still be a click
 86
 87  // Points-mode cursor (Barry, 2026-10-27): the system crosshair sits dead
 88  // center on the real cursor position, which buries the disc you're
 89  // dragging right when you need to see it land on or near another one.
 90  // This is a custom cursor instead: a dot, offset from the actual pointer
 91  // position, joined to it by a thin line. The "2 2" after the url() is the
 92  // hotspot -- the pixel of the SVG that the browser treats as the real
 93  // cursor position -- and (2,2) is blank, so nothing is drawn over the
 94  // point you're actually pointing at. CURSOR_OFFSET is how far away the
 95  // visible dot sits; 14px clears a disc's own 9px radius plus its selected
 96  // halo. Tune by eye and re-measure against MARKER_RADIUS/HALO_GAP in
 97  // figurePoints.js if those ever change.
 98  const CURSOR_OFFSET = 14;
 99  const CURSOR_HOT = 2;
100  const CURSOR_SVG =
101    `<svg xmlns="http://www.w3.org/2000/svg" width="32" height="32">` +
102    `<line x1="${CURSOR_HOT}" y1="${CURSOR_HOT}" x2="${CURSOR_HOT + CURSOR_OFFSET}" y2="${CURSOR_HOT +
CURSOR_OFFSET}" ` +
103    `stroke="#1565c0" stroke-width="1.5"/>` +
104    `<circle cx="${CURSOR_HOT + CURSOR_OFFSET}" cy="${CURSOR_HOT + CURSOR_OFFSET}" r="4" ` +
105    `fill="#1565c0" stroke="white" stroke-width="1"/>` +
106    `</svg>`;
107  const POINTS_CURSOR =
108    `url("data:image/svg+xml,${encodeURIComponent(CURSOR_SVG)}") ${CURSOR_HOT} ${CURSOR_HOT},
crosshair`;
109
110  let controller = null;   // AbortController of the active listeners
111
112
113  /* ------------------------------------------------------------
114     showWeldNotice(message)
115     A short-lived note in the caption bar (Phase 8: weld created,
116     removed, or nothing nearby to weld). Self-clearing, so it never
117     needs to be remembered to be cleaned up.
118     ------------------------------------------------------------ */
119  function showWeldNotice(message) {
120    const caption = document.getElementById("caption");
121    if (!caption) return;
```

```
122
123   let el = document.getElementById("figureWeldNotice");
124   if (!el) {
125     el = document.createElement("span");
126     el.id = "figureWeldNotice";
127     el.style.cssText = "margin-left:12px; font-style:italic; opacity:0.85;";
128     caption.appendChild(el);
129   }
130   el.textContent = message;
131   clearTimeout(el._weldTimer);
132   el._weldTimer = setTimeout(() => { el.textContent = ""; }, 2500);
133 } // end showWeldNotice
134
135
136 /* ============================================================
137    activatePointsInput(tabId, redraw)
138    ------------------------------------------------------------
139    redraw() redraws the whole figure. Replaces any earlier
140    activation. Called after armInteractor() (renderLayerPanel in
141    figureLayers.js does this): both use the same interaction layer,
142    the interactor draws the handles on it, and an object with no
143    handles makes the interactor hide that layer, which has to be
144    shown again here.
145
146    How the listeners are organized: every listener is registered with
147    { signal } from one AbortController. deactivatePointsInput() calls
148    controller.abort(), which removes all of them at once -- no list of
149    listeners to keep and take down one by one.
150
151    The mouse state between press and release lives in four variables
152    (pressedAt, dragged, moved, onHandleAtPress). They are local to this
153    activation, so a new activation starts clean.
154    ============================================================ */
155 export function activatePointsInput(tabId, redraw) {
156
157   deactivatePointsInput();
158
159   const layer = canvasLayerManager.get("interaction");
160   const main  = window.drawCanvas;
161   if (!layer || !main) return;
162
163   // Resizing a canvas clears it, and the interactor may already have drawn
164   // the handles here, so only resize when the size really differs.
165   if (layer.width  !== main.width)  layer.width  = main.width;
166   if (layer.height !== main.height) layer.height = main.height;
167   layer.style.display       = "block";
168   layer.style.pointerEvents = "auto";
169
170   // True when the active layer is visible, so a disc placed on it can be seen.
171   const editable = () => {
172     const f = uiState.figures.tabs[tabId]?.figure;
173     return !!f && !!f.layers[f.activeLayer]?.visible;
174   };
175   layer.style.cursor = editable() ? POINTS_CURSOR : "not-allowed";
176
177   controller = new AbortController();
178   const { signal } = controller;
179
180   const tabState = () => uiState.figures.tabs[tabId];
181
182   // Mouse position in canvas pixels. The canvas may be shown larger or
183   // smaller than its own pixel size (CSS scaling), so the offset from
184   // its corner is scaled by width / shown width.
185   const coords = (e) => {
186     const rect = layer.getBoundingClientRect();
```

```
    return {
      x: (e.clientX - rect.left) * (layer.width  / rect.width),
      y: (e.clientY - rect.top)  * (layer.height / rect.height)
    };
  };

  // True when (x, y) is within grab range of a handle of the active
  // object, the same range the interactor uses.
  const onHandle = (ts, x, y) => {
    const pts = ts.instances?.[ts.figure.activeLayer]?.params?.points;
    if (!pts) return false;
    const xform = layerTransformOf(ts);
    return pts.some((p) => {
      const c = applyTransform(xform, p);        // handles are stored in object space
      return Math.hypot(c.x - x, c.y - y) < PointPickerPresets.hitTolerance;
    });
  };

  // The transform of the active layer, or null.
  const layerTransformOf = (ts) => ts.figure.layers[ts.figure.activeLayer]?.transform || null;

  let pressedAt = null;    // where the mouse went down, or null
  let dragged   = null;    // the free disc being dragged, or null
  let moved     = false;   // true once the press has moved more than a click
  let onHandleAtPress = false;   // the press began on a handle of the object

  layer.addEventListener("mousedown", (e) => {
    // Left button only. A right-click is the weld toggle (below); without
    // this it would also run as a click on release and place or select a
    // disc before the weld code looks (on Windows the contextmenu event
    // comes after mouseup).
    if (e.button !== 0) return;

    const ts = tabState();
    if (!ts) return;

    pressedAt = coords(e);
    moved     = false;
    dragged   = null;

    onHandleAtPress = onHandle(ts, pressedAt.x, pressedAt.y);
    if (onHandleAtPress) return;               // the interactor's

    const hit = findPoint(ts.figure, pressedAt.x, pressedAt.y, uiState.figures.hitRadius);
    if (hit && hit.kind === "free") dragged = hit;
  }, { signal });

  // Listens on window, not the layer, so a drag that strays off the
  // canvas is still followed. Until the press has moved more than
  // CLICK_SLOP it is still a click; after that only a free disc moves.
  window.addEventListener("mousemove", (e) => {
    if (!pressedAt) return;
    const m = coords(e);
    if (!moved && Math.hypot(m.x - pressedAt.x, m.y - pressedAt.y) <= CLICK_SLOP) return;
    moved = true;

    if (!dragged) return;
    const ts = tabState();
    if (!ts) return;
    moveFreePoint(dragged, m.x, m.y, layerTransformOf(ts));
    ts.selectedPoint = dragged.n;
    redraw();
  }, { signal });

  // The release decides what the press was. In order:
```

```
  //   pressed on a free disc         -> select it (it may have been dragged)
  //   dragged a handle               -> nothing more (the interactor moved it)
  //   on a disc of the active layer  -> select it
  //   near a candidate               -> number an anchored disc there
  //   anywhere else                  -> a free disc where the press began
  // A right press never gets here: mousedown left pressedAt null.
  window.addEventListener("mouseup", (e) => {
    if (!pressedAt) return;
    const wasDrag  = moved;
    const wasFree  = dragged;
    const m        = pressedAt;              // a click happens where it went down
    pressedAt = null;
    dragged   = null;

    const ts = tabState();
    if (!ts) return;
    if (!editable()) return;                 // hidden active layer

    if (wasFree) {                           // on a free disc (maybe dragged): select it
      ts.selectedPoint = wasFree.n;
      redraw();
      return;
    }
    if (wasDrag && onHandleAtPress) return;   // the interactor dragged a handle

    const radius = uiState.figures.hitRadius;
    const figure = ts.figure;

    let point = findPoint(figure, m.x, m.y, radius);

    if (!point) {
      const c = findCandidate(ts, m.x, m.y, radius);
      point = c ? addAnchoredPoint(figure, c)
                : addFreePoint(figure, m.x, m.y, layerTransformOf(ts));
    }

    ts.selectedPoint = point.n;
    redraw();
  }, { signal });

  // Right-click: toggle the weld under the cursor (figureWelds.js).
  // Both the mousedown and the contextmenu event are suppressed, so the
  // browser's own menu never appears.
  layer.addEventListener("mousedown", (e) => {
    if (e.button === 2) e.preventDefault();
  }, { signal });

  layer.addEventListener("contextmenu", (e) => {
    e.preventDefault();
    const ts = tabState();
    if (!ts || !editable()) return;

    const m = coords(e);
    const result = toggleWeldAt(ts, m.x, m.y, uiState.figures.hitRadius);
    if (result.point) ts.selectedPoint = result.point.n;
    showWeldNotice(result.message);
    redraw();
  }, { signal });

  // Delete or Backspace removes the selected disc -- but not while the
  // keyboard is in a text box, slider or list, where those keys belong
  // to the control.
  document.addEventListener("keydown", (e) => {
    if (e.key !== "Delete" && e.key !== "Backspace") return;
    if (uiState.activeTab !== "figures") return;
```

```
317
318     const tag = (e.target && e.target.tagName) || "";
319     if (tag === "INPUT" || tag === "SELECT" || tag === "TEXTAREA") return;
320
321     const ts = tabState();
322     if (!ts || ts.selectedPoint == null) return;
323
324     e.preventDefault();
325     deletePoint(ts.figure, ts.selectedPoint);
326     ts.selectedPoint = null;
327     redraw();
328   }, { signal });
329
330 } // end activatePointsInput
331
332
333 /* ============================================================
334    deactivatePointsInput()
335    ------------------------------------------------------------
336    Removes every points-mode listener (one abort) and hides the
337    interaction layer. Safe to call when nothing is active.
338    ============================================================ */
339 export function deactivatePointsInput() {
340
341   if (controller) {
342     controller.abort();
343     controller = null;
344   }
345
346   const layer = canvasLayerManager.get("interaction");
347   if (layer) {
348     layer.style.display       = "none";
349     layer.style.pointerEvents = "none";
350   }
351
352 } // end deactivatePointsInput
353
354 // end figurePointer.js
```

figureCentroid.js

ui/figures/figureCentroid.js · 372 lines

MOVE AND TURN A WHOLE LAYER

← Contents

```
/* figureCentroid.js
   ============================================================
   Figures Tab -- Centroid, Bounding Box and Rotation (Phase 7b)
   ============================================================
   Role:
     Moves and turns a whole layer. The active layer shows a small diamond
     at its centroid. Drag the diamond to move the layer. Click it to select
     it: a padded bounding box appears with one round rotation handle on a
     stem above the top edge. Drag that handle to turn the layer about the
     centroid. With the diamond selected the arrow keys move the layer 1 px
     (10 with Shift).

     Nothing in the object changes. Moving or turning only edits the
     layer's transform (layer.transform = { angle, tx, ty }, see
     draw/drawTransform.js); the object keeps drawing in its own coordinates
     and drawLine maps them. So a handle, a stitch count, a slider all keep
     meaning what they meant.

   Where things come from:
     bounds    drawLine notes every end it draws into drawState.bounds while
               a layer draws; redrawFigure keeps the result in
               instance.bounds (object space, before the transform).
     centroid  the center of those bounds. (The center of the bounding box,
               not an average of the points: it is cheap, steady while the
               shape is edited, and the same for every object.)
     box       the bounds grown by BOX_PAD on every side, turned by the
               layer's angle.

   Rules:
     * Only the active layer has a centroid, and only while it is visible.
       Show All (preview), final mode and points mode hide it. Points mode
       is left alone on purpose: the middle of a Line is both its centroid
       and the natural place for a disc, and a click must reach the disc.
     * A press on the centroid or the rotation handle wins over an object
       handle under it (centroidHit; the interactor asks).
     * Selecting a layer, hiding it, or toggling points mode deselects.
       The selection is tabState.centroidSelected (not saved).
     * Not built: scaling, snapping the angle with Shift.

   The drawing goes on the "bbox" overlay canvas, which is otherwise
   unused in Figures. The mouse is read from window, like interactor.js, so
   an object with no handles still works.

   Welds (Phase 8): moving or turning a layer moves its welded discs, so
   setTransform() below takes a snapshot of them first and lets their
   partners follow afterwards (figureWelds.js). A turn is handed to the
   partners as a plain shift of each disc, which is exact for the discs
   themselves but means a welded partner is shifted, not turned.

   Known gap (2026-10-27): the mousedown handler does not check which
   button was pressed, so a right-press on the diamond starts a move,
   the same bug that was fixed in interactor.js and figurePointer.js.
   It only matters in lines mode (the diamond is hidden in points mode,
   where right-click welds). One-line fix, listed in PHASES.md.

   A worked example of a turn: the diamond is at (300, 300), the press
   on the rotation handle at (300, 250), straight above it, so
   angle0 = atan2(-50, 0) = -90 degrees. The mouse moves to (350, 300),
```

```
   straight right: now = atan2(0, 50) = 0 degrees. The layer is turned
   by now - angle0 = +90 degrees about (300, 300): a quarter turn
   clockwise on screen.

   Exports:
     activateCentroid(tabId, redraw)
     deactivateCentroid()
     centroidHit(tabState, x, y)
     drawCentroid(tabState)
   ============================================================ */

import { canvasLayerManager } from "/ui/canvasLayerManager.js";
import { uiState }            from "/ui/uiState.js";
import { refreshInteractor }  from "/ui/interactor.js";
import { drawState }          from "/draw/drawState.js";
import {
  newTransform,
  applyTransform,
  translateTransform,
  rotateTransformAbout
} from "/draw/drawTransform.js";
import { snapshotWeldMembers, propagateAfterChange } from "./figureWelds.js";

const BOX_PAD     = 6;     // box margin around the bounds, canvas pixels
const STEM        = 26;    // from the box's top edge up to the rotation handle
const DIAMOND     = 7;     // half-size of the centroid diamond
const HANDLE_R    = 6;     // radius of the rotation handle
const HIT_RADIUS  = 9;     // grab range of the diamond and of the handle
const NUDGE       = 1;     // arrow key step; Shift makes it 10
const NUDGE_SHIFT = 10;
const COLOR       = "#1565c0";
const COLOR_ON    = "#FE8A18";   // selected (same orange as an active handle)

let controller = null;   // AbortController of the active listeners


/* ------------------------------------------------------------
   geometry(tabState)
   Where everything is, in canvas pixels, or null when the active layer has
   no centroid right now (none, hidden, nothing drawn, Show All).
   Returns { center, corners[4], stemFrom, handle }.
   ------------------------------------------------------------ */
function geometry(ts) {

  if (!ts || !ts.figure || ts.preview || drawState.final) return null;
  if (ts.mode === "points") return null;     // see "Rules" in the header

  const figure = ts.figure;
  const layer  = figure.layers[figure.activeLayer];
  const inst   = ts.instances?.[figure.activeLayer];
  const b      = inst?.bounds;
  if (!layer || !layer.visible || !b || !isFinite(b.minX)) return null;

  // Everything is worked out in object space (where the bounds are),
  // then each spot is sent to the canvas through the layer's transform.
  // That is why the box turns with the layer instead of staying square
  // to the screen.
  const t  = layer.transform || null;
  const cx = (b.minX + b.maxX) / 2;
  const cy = (b.minY + b.maxY) / 2;
  const x0 = b.minX - BOX_PAD, x1 = b.maxX + BOX_PAD;
  const y0 = b.minY - BOX_PAD, y1 = b.maxY + BOX_PAD;

  return {
    center:   applyTransform(t, { x: cx, y: cy }),
```

```
    corners:  [
      applyTransform(t, { x: x0, y: y0 }),
      applyTransform(t, { x: x1, y: y0 }),
      applyTransform(t, { x: x1, y: y1 }),
      applyTransform(t, { x: x0, y: y1 })
    ],
    stemFrom: applyTransform(t, { x: cx, y: y0 }),
    handle:   applyTransform(t, { x: cx, y: y0 - STEM })
  };

} // end geometry


/* ============================================================
   centroidHit(tabState, x, y)
   ------------------------------------------------------------
   True when canvas point (x, y) is on the diamond, or on the rotation
   handle while the diamond is selected.
   ============================================================ */
export function centroidHit(ts, x, y) {

  const g = geometry(ts);
  if (!g) return false;

  if (Math.hypot(g.center.x - x, g.center.y - y) < HIT_RADIUS) return true;
  return !!ts.centroidSelected && Math.hypot(g.handle.x - x, g.handle.y - y) < HIT_RADIUS;

} // end centroidHit


/* ============================================================
   drawCentroid(tabState)
   ------------------------------------------------------------
   Clears the overlay and draws the diamond (and, when selected, the box,
   stem and rotation handle) of the active layer.
   ============================================================ */
export function drawCentroid(ts) {

  const canvas = canvasLayerManager.get("bbox");
  if (!canvas) return;
  const c = canvas.getContext("2d");
  c.clearRect(0, 0, canvas.width, canvas.height);

  const g = geometry(ts);
  if (!g) return;

  const on = !!ts.centroidSelected;

  if (on) {
    c.save();
    c.strokeStyle = COLOR;
    c.lineWidth   = 1;
    c.setLineDash([5, 4]);
    c.beginPath();
    g.corners.forEach((p, i) => (i === 0 ? c.moveTo(p.x, p.y) : c.lineTo(p.x, p.y)));
    c.closePath();
    c.stroke();
    c.setLineDash([]);

    c.beginPath();                       // stem
    c.moveTo(g.stemFrom.x, g.stemFrom.y);
    c.lineTo(g.handle.x, g.handle.y);
    c.stroke();

    c.beginPath();                       // rotation handle
```

```
    c.arc(g.handle.x, g.handle.y, HANDLE_R, 0, Math.PI * 2);
    c.fillStyle = "white";
    c.fill();
    c.lineWidth = 2;
    c.stroke();
    c.restore();
  }

  c.save();                              // the diamond, always
  c.beginPath();
  c.moveTo(g.center.x,           g.center.y - DIAMOND);
  c.lineTo(g.center.x + DIAMOND, g.center.y);
  c.lineTo(g.center.x,           g.center.y + DIAMOND);
  c.lineTo(g.center.x - DIAMOND, g.center.y);
  c.closePath();
  c.fillStyle   = on ? COLOR_ON : COLOR;
  c.fill();
  c.strokeStyle = "white";
  c.lineWidth   = 1.5;
  c.stroke();
  c.restore();

} // end drawCentroid


/* ============================================================
   activateCentroid(tabId, redraw)
   ------------------------------------------------------------
   Starts listening. redraw() redraws the whole figure (which also redraws
   the guides). Replaces any earlier activation.
   ============================================================ */
export function activateCentroid(tabId, redraw) {

  deactivateCentroid();

  controller = new AbortController();
  const { signal } = controller;

  const tabState = () => uiState.figures.tabs[tabId];

  // Only while this figure is the one on screen.
  const live = () =>
    uiState.activeTab === "figures" &&
    uiState.figures.activeSubtab === tabId &&
    !!tabState()?.figure;

  const coords = (e) => {
    const main = window.drawCanvas;
    const rect = main.getBoundingClientRect();
    return {
      x:  (e.clientX - rect.left) * (main.width  / rect.width),
      y:  (e.clientY - rect.top)  * (main.height / rect.height),
      in: e.clientX >= rect.left && e.clientX <= rect.right &&
          e.clientY >= rect.top  && e.clientY <= rect.bottom
    };
  };

  // The layer being moved or turned.
  const activeLayer = (ts) => ts.figure.layers[ts.figure.activeLayer];

  // Writes a transform into the layer without replacing the object, so
  // everything holding the layer sees it. Phase 8: the layer's welded
  // discs are snapshotted first, then let follow once redraw() has
  // moved everything and refreshed the points.
  const setTransform = (ts, t) => {
```

```
    const layer      = activeLayer(ts);
    const layerIndex = ts.figure.activeLayer;
    const snapshot    = snapshotWeldMembers(ts.figure, layerIndex);
    if (!layer.transform) layer.transform = newTransform();
    Object.assign(layer.transform, t);
    redraw();
    refreshInteractor();
    propagateAfterChange(tabId, layerIndex, snapshot);
  };

  let drag = null;   // { kind: "move"|"turn", from: {x,y}, start: transform, center, angle0 }

  // Every move is computed from where the drag STARTED (drag.start and
  // drag.from), not from the previous mouse position. Small rounding
  // errors therefore never pile up over a long drag.
  //
  // Press order of precedence: the rotation handle (only while
  // selected), then the diamond, then anywhere else on the canvas,
  // which deselects.
  window.addEventListener("mousedown", (e) => {
    if (!live()) return;
    const ts = tabState();
    const m  = coords(e);
    const g  = geometry(ts);

    if (g && ts.centroidSelected &&
        Math.hypot(g.handle.x - m.x, g.handle.y - m.y) < HIT_RADIUS) {
      drag = {
        kind:   "turn",
        start:  { ...(activeLayer(ts).transform || newTransform()) },
        center: g.center,
        angle0: Math.atan2(m.y - g.center.y, m.x - g.center.x)
      };
      return;
    }

    if (g && Math.hypot(g.center.x - m.x, g.center.y - m.y) < HIT_RADIUS) {
      drag = {
        kind:  "move",
        from:  m,
        start: { ...(activeLayer(ts).transform || newTransform()) }
      };
      if (!ts.centroidSelected) {
        ts.centroidSelected = true;
        drawCentroid(ts);
      }
      return;
    }

    // A press anywhere else on the canvas deselects.
    if (m.in && ts.centroidSelected) {
      ts.centroidSelected = false;
      drawCentroid(ts);
    }
  }, { signal });

  window.addEventListener("mousemove", (e) => {
    if (!drag || !live()) return;
    const ts = tabState();
    const m  = coords(e);

    if (drag.kind === "move") {
      setTransform(ts, translateTransform(drag.start, m.x - drag.from.x, m.y - drag.from.y));
    } else {
      // Math.atan2(dy, dx) is the direction from the centroid to the mouse,
```

```
      // as an angle. The layer turns by how far that direction has swung
      // since the press (see the worked example in the header).
      const now = Math.atan2(m.y - drag.center.y, m.x - drag.center.x);
      setTransform(ts, rotateTransformAbout(drag.start, drag.center.x, drag.center.y, now -
drag.angle0));
    }
  }, { signal });

  window.addEventListener("mouseup", () => {
    drag = null;
  }, { signal });

  // Arrow keys move the layer while the diamond is selected.
  window.addEventListener("keydown", (e) => {
    if (!live()) return;
    const ts = tabState();
    if (!ts.centroidSelected || !geometry(ts)) return;

    const tag = (e.target && e.target.tagName) || "";
    if (tag === "INPUT" || tag === "SELECT" || tag === "TEXTAREA") return;

    const step = e.shiftKey ? NUDGE_SHIFT : NUDGE;
    let dx = 0;
    let dy = 0;
    switch (e.key) {
      case "ArrowLeft":  dx = -step; break;
      case "ArrowRight": dx =  step; break;
      case "ArrowUp":    dy = -step; break;
      case "ArrowDown":  dy =  step; break;
      default: return;
    }

    e.preventDefault();
    setTransform(ts, translateTransform(activeLayer(ts).transform, dx, dy));
  }, { signal });

} // end activateCentroid


/* ============================================================
   deactivateCentroid()
   ------------------------------------------------------------
   Removes all four listeners with one abort. Safe to call when
   nothing is active.
   ============================================================ */
export function deactivateCentroid() {

  if (controller) {
    controller.abort();
    controller = null;
  }

} // end deactivateCentroid

// end figureCentroid.js
```

figureWelds.js

ui/figures/figureWelds.js · 580 lines

WELDS

← Contents

```
/* figureWelds.js
   ============================================================
   Figures Tab -- Welds (Phase 8)
   ============================================================
   Role:
     A weld ties together discs that sit at the same spot on different
     layers (or, for a corner, two discs on the same layer). A weld is
     not itself draggable: dragging one of its member discs (because the
     disc is on a handle, or is a free disc) moves every other member's
     layer to follow, so the figure keeps meeting at that point.

   Data (figure.welds, not yet saved -- Phase 10):
     figure.welds = [ { members: [ { layer, n }, ... ] } ]
     members   the discs in the group, by their figure.points number.
     Whether a group is satisfied (its members still coincide) is not
     stored at all. drawWeldMarks() works it out on every draw and
     shows an unsatisfied group with a red mark instead of a black one.
     (An earlier version of this comment listed an "unsatisfied" field;
     the code never had one.)
     figure.welds does not exist until the first weld is made
     (createOrJoinWeld creates it), so readers write figure.welds || [].

   Creating and removing a weld (right-click, figurePointer.js):
     toggleWeldAt() is the whole of it. Right-click a disc already in a
     group to drop it out (dissolving the group if that leaves fewer
     than two members). Right-click a disc not in a group: if another
     disc (any layer, the same layer included, for tying a corner) sits
     within hit radius, join or start a group with it. Right-click a
     bare infrastructure point: numbers it first (the same as a left
     click would), but only if that creates a disc that lands on an
     existing one, or on another arm of its own object at the same
     spot -- otherwise nothing happens, so right-clicking empty
     infrastructure cannot litter the figure with lone discs.
     Creating a weld never moves anything: discs must already coincide.

     cornerPartners(): an object's own corner handles (a Parabola's
     line1_pt2 and line2_pt1, say) are separate fields, independent by
     default (Phase 7's "fan/bowtie" finding) -- only one of them is
     ever the disc a corner click lands on (findCandidate's own tie-
     break always prefers the first arm), so welding just that one to
     something else leaves the object's OTHER corner handle behind,
     splitting the object's own corner the moment the weld moves it.
     toggleWeldAt() calls cornerPartners() on every weld it makes, so
     reaching another arm's matching candidate at the same spot numbers
     it too and pulls it into the same group -- this is what actually
     makes "two members on the same layer, for a corner" (the spec's
     own words) reachable from a single right-click, not just a shape
     the data model happens to allow.

   Moving a weld (propagateAfterChange(), the move-propagation rule):
     Hooked into moveHandle() (ui/interactor.js, drag and the arrow
     keys) and setTransform() (figureCentroid.js, move and turn) via a
     snapshot taken just before the change and compared just after.
     Every weld member of the layer being changed that actually moved
     seeds a breadth-first walk of the weld graph. For each neighbor
     disc reached:
       - the neighbor sits exactly at one of its own object's handles
         (t = 0 or 1, and that arm end matches a handle) -> move that
```

```
          one handle by the same canvas delta;
        - the neighbor is mid-arm and BOTH ends of that specific arm are
          handles (Line, Parabola, Radiate) -> move both, so the arm
          reshapes and the object's other handles stay still;
        - anywhere else (the arm's ends are not handles at all, as for
          Regular Polygon or a node ring) -> translate the whole layer.
      Each handle moves at most once per drag (two different handles of
      one layer can each move once, which is how a corner's two members
      travel together); a second weld edge that would move the same
      handle again, or a whole layer that has already moved in pieces or
      as a whole, is left alone, which is how an unsatisfiable weld
      (two paths disagreeing about where something should land) shows up.

    A worked example. Layer 1 is a Parabola with its far end
    (line2_pt2) at (350, 350); layer 0 is a Line whose start (pt1) is
    also at (350, 350). Disc 2 sits on the Parabola's arm2 at t = 1,
    disc 1 on the Line at t = 0, and figure.welds is
      [ { members: [ { layer: 0, n: 1 }, { layer: 1, n: 2 } ] } ]
    You drag the Line's start to (410, 310):
      1. interactor.js calls beforeMove: snapshot { 1 -> (350, 350) }
         (only layer 0's welded discs; the Line is the layer moving).
      2. the handle moves, the Line redraws, refreshPoints puts disc 1
         at (410, 310).
      3. afterMove calls propagateAfterChange: disc 1 moved by
         (+60, -40), so it seeds the walk.
      4. its weld's other member is disc 2: anchored, t = 1, and the
         end of arm2 is the Parabola's own handle line2_pt2. So that
         one handle moves by (+60, -40), to (410, 310).
      5. disc 2 is queued in turn, but its weld has no member left
         that hasn't been reached, so the walk ends; one redraw.
    The Parabola's other end and its corner do not move: only the
    welded end follows, and the parabola reshapes.

    Why the weld mark looks like a ring, not a + (2026-10-27): the mark
    is drawn on the main canvas, but the handles are drawn on the
    interaction layer above it. A weld made on a handle has that
    handle's dot (radius 5) right on top of the + (arms 5 long), so
    only the white halo around it shows. Proposed fix, not built: draw
    the + larger than a handle, or on the interaction layer after the
    handles. See PHASES.md.

    Exports:
      weldOf(figure, n)
      removeFromWeld(figure, n)
      toggleWeldAt(tabState, x, y, radius)
      snapshotWeldMembers(figure, layerIndex)
      propagateAfterChange(tabId, layerIndex, snapshot)
      drawWeldMarks(tabState)
    ============================================================ */

import { uiState } from "/ui/uiState.js";
import { applyTransform, inverseTransform, translateTransform } from "/draw/drawTransform.js";
import { findCandidate, addAnchoredPoint, moveFreePoint, refreshPoints, armsOf, armPoint } from
"./figurePoints.js";
// figureLayers.js imports this module too (for snapshotWeldMembers,
// propagateAfterChange and drawWeldMarks); importing redrawFigure back
// from it here is a circular import, but it is fine -- this binding is
// only used inside a function body, well after both modules have
// finished loading.
import { redrawFigure } from "./figureLayers.js";

const EPS = 0.5;         // canvas pixels; "the same spot" and "didn't move"
const MARK_RADIUS = 7;   // the plus mark's white halo
const MARK_ARM    = 5;   // the plus mark's arm length

```

```
123
124 /* ------------------------------------------------------------
125    pointByN(figure, n)
126    ------------------------------------------------------------ */
127 function pointByN(figure, n) {
128   return figure.points.find((p) => p.n === n) || null;
129 } // end pointByN
130
131
132 /* ============================================================
133    weldOf(figure, n)
134    ------------------------------------------------------------
135    The weld group point n belongs to, or null.
136    ============================================================ */
137 export function weldOf(figure, n) {
138   const list = figure.welds || [];
139   return list.find((w) => w.members.some((m) => m.n === n)) || null;
140 } // end weldOf
141
142
143 /* ------------------------------------------------------------
144    createOrJoinWeld(figure, nA, nB)
145    Joins the two points into one weld group, merging their groups if
146    each already had one. Does not check coincidence -- the caller
147    (toggleWeldAt) only calls this once it has found two discs at the
148    same spot.
149    ------------------------------------------------------------ */
150 function createOrJoinWeld(figure, nA, nB) {
151   if (!figure.welds) figure.welds = [];
152   const pa = pointByN(figure, nA);
153   const pb = pointByN(figure, nB);
154   if (!pa || !pb) return null;
155
156   const wa = weldOf(figure, nA);
157   const wb = weldOf(figure, nB);
158
159   if (wa && wb) {
160     if (wa === wb) return wa;
161     wa.members.push(...wb.members);
162     figure.welds = figure.welds.filter((w) => w !== wb);
163     return wa;
164   }
165   if (wa) { wa.members.push({ layer: pb.layer, n: pb.n }); return wa; }
166   if (wb) { wb.members.push({ layer: pa.layer, n: pa.n }); return wb; }
167
168   const weld = { members: [
169     { layer: pa.layer, n: pa.n },
170     { layer: pb.layer, n: pb.n }
171   ] };
172   figure.welds.push(weld);
173   return weld;
174 } // end createOrJoinWeld
175
176
177 /* ============================================================
178    removeFromWeld(figure, n)
179    ------------------------------------------------------------
180    Drops point n out of its weld group, dissolving the group if that
181    leaves fewer than two members. True if n was welded. Also called
182    (figurePoints.js) whenever a point disappears for some other reason
183    -- deleted by the person, or its anchor arm has gone -- since a
184    vanished member leaves its weld the same way an explicit unweld does.
185    ============================================================ */
186 export function removeFromWeld(figure, n) {
187   const weld = weldOf(figure, n);
```

```
  if (!weld) return false;
  weld.members = weld.members.filter((m) => m.n !== n);
  if (weld.members.length < 2) {
    figure.welds = figure.welds.filter((w) => w !== weld);
  }
  return true;
} // end removeFromWeld


/* ------------------------------------------------------------
   cornerPartners(tabState, point)
   ------------------------------------------------------------
   A Parabola's two corner handles (line1_pt2 and line2_pt1, say) are
   two separate fields, independent by default (Phase 7's "fan/bowtie"
   finding): they only start out at the same spot, nothing keeps them
   there. So a weld made on just one of them does not, by itself, tie
   the object's own corner together -- the spot found here is arm1's
   end; arm2's start is a different field entirely, untouched by
   anything that moves arm1's. This finds the other arm(s) of the
   SAME layer that also have a candidate sitting exactly on point's
   spot, numbering one there (figurePoints.js already has a disc there
   only by coincidence, never by sharing a number) if none exists yet.
   Returns the discs found, so the caller can weld them in too -- this
   is what actually makes "weld a corner" (same-layer members) reachable
   from a single right-click, since right-clicking an already-anchored
   spot otherwise only ever finds the ONE disc already sitting there.
   ------------------------------------------------------------ */
function cornerPartners(tabState, point) {

  // Only an anchored disc sits on an arm of its object, so only it can be
  // at a corner. A free disc has no arm: without this check every arm of
  // its layer looked like "another arm", and a free disc lying on its own
  // object's handle was "corner tied" to it.
  if (point.kind !== "anchored") return [];

  const figure   = tabState.figure;
  const instance = tabState.instances?.[point.layer];
  if (!instance) return [];

  const xform = figure.layers[point.layer]?.transform || null;
  const found = [];

  for (const arm of armsOf(instance)) {
    if (arm.id === point.arm) continue;

    for (let i = arm.from; i <= arm.to; i++) {
      const t  = i / arm.steps;
      const pt = armPoint(arm, t, xform);
      if (Math.hypot(pt.x - point.x, pt.y - point.y) > EPS) continue;

      let disc = figure.points.find((p) =>
        p.layer === point.layer && p.arm === arm.id && Math.abs(p.t - t) < 1e-9);
      if (!disc) disc = addAnchoredPoint(figure, { layer: point.layer, arm: arm.id, t, x: pt.x, y:
pt.y });
      found.push(disc);
      break;   // one candidate on this arm is enough
    }
  }

  return found;
} // end cornerPartners


/* ============================================================
   toggleWeldAt(tabState, x, y, radius)
```

```
    ------------------------------------------------------------
    The right-click action. Returns { kind: "weld"|"unweld"|"none",
    message, point? }.
    ============================================================ */
export function toggleWeldAt(tabState, x, y, radius) {

  const figure = tabState.figure;

  // The disc nearest (px, py) within radius, on ANY visible layer (not
  // just the active one -- a weld joins layers), optionally skipping one
  // disc by number. Used twice: for the disc under the cursor, then for
  // a partner sitting on that disc.
  const discAt = (px, py, excludeN = null) => {
    let best = null, bestDist = radius;
    for (const p of figure.points) {
      if (p.n === excludeN) continue;
      const layer = figure.layers[p.layer];
      if (!layer || !layer.visible) continue;
      const d = Math.hypot(p.x - px, p.y - py);
      if (d < bestDist) { bestDist = d; best = p; }
    }
    return best;
  };

  let clicked = discAt(x, y);

  if (!clicked) {
    // No disc there yet. A bare infrastructure point is only numbered
    // if doing so lands it on a disc that is already there, or on
    // another arm of its own object's candidates at the same spot (a
    // Parabola corner reached from its empty side) -- otherwise there
    // is nothing to weld, and nothing is created.
    const candidate = findCandidate(tabState, x, y, radius);
    if (!candidate) return { kind: "none", message: "Nothing to weld here." };

    const partner = discAt(candidate.x, candidate.y);
    const probe   = { kind: "anchored", layer: candidate.layer, arm: candidate.arm, x: candidate.x,
y: candidate.y };
    if (!partner && !cornerPartners(tabState, probe).length) {
      return { kind: "none", message: "Nothing to weld here." };
    }

    clicked = addAnchoredPoint(figure, candidate);
  }

  if (weldOf(figure, clicked.n)) {
    removeFromWeld(figure, clicked.n);
    return { kind: "unweld", point: clicked, message: "Weld removed." };
  }

  // Same-layer partners are allowed on purpose: that is how two corner
  // handles of one object get tied together. corners finds them even
  // when clicked is the only one of them with a disc of its own yet.
  const partner = discAt(clicked.x, clicked.y, clicked.n);
  const corners = cornerPartners(tabState, clicked);

  if (!partner && !corners.length) {
    return { kind: "none", point: clicked, message: "Nothing to weld here." };
  }

  if (partner) createOrJoinWeld(figure, clicked.n, partner.n);
  for (const c of corners) createOrJoinWeld(figure, clicked.n, c.n);

  return {
    kind: "weld", point: clicked,
```

```
    message: corners.length ? "Welded (corner tied)." : "Welded."
  };

} // end toggleWeldAt


/* ============================================================
   snapshotWeldMembers(figure, layerIndex)
   ------------------------------------------------------------
   The current canvas position of every welded disc on one layer,
   taken just before that layer is about to change. { n -> {x,y} }.
   ============================================================ */
export function snapshotWeldMembers(figure, layerIndex) {
  const snap = new Map();
  if (!figure.welds || !figure.welds.length) return snap;

  for (const weld of figure.welds) {
    for (const m of weld.members) {
      if (m.layer !== layerIndex) continue;
      const p = pointByN(figure, m.n);
      if (p) snap.set(p.n, { x: p.x, y: p.y });
    }
  }
  return snap;
} // end snapshotWeldMembers


/* ------------------------------------------------------------
   findHandleIndex(instance, pt)
   The index in instance.params.points whose object-space position
   matches pt, or -1. Used to tell a real handle apart from a node an
   object only derives (Regular Polygon's, say).
   ------------------------------------------------------------ */
// Matching by position (to a millionth of a pixel) rather than by name
// works for every object without a table of field names: an arm's a and
// b are the object's own params (pt1, line2_pt2 ...), and the handles
// array is kept equal to them by the object's update().
function findHandleIndex(instance, pt) {
  const pts = instance?.params?.points || [];
  for (let i = 0; i < pts.length; i++) {
    if (Math.abs(pts[i].x - pt.x) < 1e-6 && Math.abs(pts[i].y - pt.y) < 1e-6) return i;
  }
  return -1;
} // end findHandleIndex


/* ------------------------------------------------------------
   moveFollowerHandle(instance, idx, xform, dx, dy)
   Moves one handle by a canvas-space delta and re-syncs the object.
   ------------------------------------------------------------ */
// The delta is in canvas pixels but handles are stored in object space,
// so: handle to canvas, add the delta, back to object space. For a layer
// that has been turned, this is what makes a sideways drag of the partner
// come out sideways on screen.
function moveFollowerHandle(instance, idx, xform, dx, dy) {
  const obj = instance.params.points[idx];
  const canvasPt = applyTransform(xform, obj);
  const moved = inverseTransform(xform, canvasPt.x + dx, canvasPt.y + dy);
  instance.params.points[idx] = { x: moved.x, y: moved.y };
  if (typeof instance.update === "function") instance.update(instance.params);
  return true;
} // end moveFollowerHandle


/* ------------------------------------------------------------
```

```
    translateWholeLayer(figure, layerIndex, dx, dy)
    The fallback for a node with no real handle of its own.
    ------------------------------------------------------------ */
function translateWholeLayer(figure, layerIndex, dx, dy) {
  const layer = figure.layers[layerIndex];
  if (!layer) return false;
  layer.transform = translateTransform(layer.transform, dx, dy);
  return true;
} // end translateWholeLayer


/* ------------------------------------------------------------
    applyFollowerMove(tabState, point, dx, dy, moved)
    Moves one follower disc's layer by the canvas delta, choosing
    handle-move, arm-translate, or whole-layer-translate as described
    in the header. True if something moved.

    moved tracks what has already been done to each layer THIS drag, at
    the right grain to tell a real conflict from two independent moves
    that just happen to land on the same layer: moved.get(layerIndex) =
    { whole: bool, handles: Set<index> }. A second whole-layer-translate
    of the same layer is refused (that IS a conflict -- the unsatisfiable
    case), and so is moving the same handle index twice, but two
    DIFFERENT handle indices on one layer are both allowed -- that is
    exactly how a weld ties two of an object's own corner handles
    together (cornerPartners(), above) and moves them as one.
    ------------------------------------------------------------ */
function applyFollowerMove(tabState, point, dx, dy, moved) {

  const figure = tabState.figure;
  const xform  = figure.layers[point.layer]?.transform || null;

  if (point.kind === "free") {
    // Not a layer move at all -- its own point, independent of anything
    // else on its layer -- so there is no conflict to guard here.
    return moveFreePoint(point, point.x + dx, point.y + dy, xform);
  }

  let layerMoved = moved.get(point.layer);
  if (!layerMoved) { layerMoved = { whole: false, handles: new Set() }; moved.set(point.layer,
layerMoved); }
  if (layerMoved.whole) return false;   // already moved as a whole; nothing more to add

  const instance = tabState.instances?.[point.layer];
  if (!instance || typeof instance.infrastructure !== "function") return false;
  const arm = instance.infrastructure().find((a) => a.id === point.arm);
  if (!arm) return false;

  const applyWhole = () => {
    if (layerMoved.handles.size) return false;   // handles already moved piecemeal; a whole move
now would double them
    if (!translateWholeLayer(figure, point.layer, dx, dy)) return false;
    layerMoved.whole = true;
    return true;
  };
  const applyHandle = (idx) => {
    if (layerMoved.handles.has(idx)) return false;   // this exact handle already moved this pass
    if (!moveFollowerHandle(instance, idx, xform, dx, dy)) return false;
    layerMoved.handles.add(idx);
    return true;
  };

  // An arc or ring (Phase 7d's explicit-points arm shape) has no handle
  // at either end to move on its own.
  if (arm.points) return applyWhole();
```

```
444
445   const atStart = Math.abs(point.t) < 1e-9;
446   const atEnd   = Math.abs(point.t - 1) < 1e-9;
447
448   if (atStart || atEnd) {
449     const idx = findHandleIndex(instance, atStart ? arm.a : arm.b);
450     if (idx >= 0) return applyHandle(idx);
451     return applyWhole();
452   }
453
454   // Mid-arm: the arm translates (its own two ends move) if both ends
455   // are real handles, so the object's other handles stay (provisional
456   // rule, Phase 8 spec).
457   const idxA = findHandleIndex(instance, arm.a);
458   const idxB = findHandleIndex(instance, arm.b);
459   if (idxA >= 0 && idxB >= 0) {
460     const okA = applyHandle(idxA);
461     const okB = applyHandle(idxB);
462     return okA || okB;
463   }
464
465   return applyWhole();
466
467 } // end applyFollowerMove
468
469
470 /* ============================================================
471    propagateAfterChange(tabId, layerIndex, snapshot)
472    ------------------------------------------------------------
473    Compares the welded discs of layerIndex against snapshot (from
474    snapshotWeldMembers, taken before the change) and walks the weld
475    graph outward for every one that moved. Redraws once at the end if
476    anything followed. A layer moves at most once per call.
477    ============================================================ */
478 export function propagateAfterChange(tabId, layerIndex, snapshot) {
479
480   const tabState = uiState.figures.tabs[tabId];
481   if (!tabState || !snapshot || !snapshot.size) return;
482   const figure = tabState.figure;
483
484   // What has already been done to each layer this call -- see
485   // applyFollowerMove's own header for why this is per-handle, not just
486   // per-layer. The driving layer itself counts as fully spoken for
487   // (whole: true blocks it from being touched again by a path that
488   // loops back to it), since the user's own drag already moved it.
489   // A Map is a lookup table: moved.get(layer) / moved.set(layer, value).
490   // queue is the work list of discs whose move still has to be passed
491   // on (first in, first out: a breadth-first walk). queued remembers
492   // every disc already reached, so a loop of welds cannot go round
493   // forever.
494   const moved = new Map([[layerIndex, { whole: true, handles: new Set() }]]);
495   const queue = [];
496   const queued = new Set();
497
498   for (const [n, before] of snapshot) {
499     const p = pointByN(figure, n);
500     if (!p) continue;
501     const dx = p.x - before.x, dy = p.y - before.y;
502     if (Math.hypot(dx, dy) > EPS) { queue.push({ n, dx, dy }); queued.add(n); }
503   }
504
505   let followed = false;
506
507   while (queue.length) {
508     const { n, dx, dy } = queue.shift();
```

```
    const weld = weldOf(figure, n);
    if (!weld) continue;

    for (const m of weld.members) {
      if (m.n === n || queued.has(m.n)) continue;
      const mp = pointByN(figure, m.n);
      if (!mp) continue;

      if (!applyFollowerMove(tabState, mp, dx, dy, moved)) continue;

      queued.add(m.n);
      followed = true;
      queue.push({ n: m.n, dx, dy });   // chain onward, same delta
    }
  }

  if (followed) {
    refreshPoints(tabState);
    redrawFigure(tabId);
  }

} // end propagateAfterChange


/* ============================================================
   drawWeldMarks(tabState)
   ------------------------------------------------------------
   One mark per weld group, at any one member's position (all of a
   satisfied group's members coincide): a black plus on a white halo,
   or red while the group's members have drifted apart (unsatisfiable
   this drag). Figured fresh every call, nothing is stored.
   ============================================================ */
export function drawWeldMarks(tabState) {

  const figure = tabState.figure;
  if (!figure.welds || !figure.welds.length) return;

  for (const weld of figure.welds) {
    const pts = weld.members.map((m) => pointByN(figure, m.n)).filter(Boolean);
    if (!pts.length) continue;

    const visible = pts.some((p) => figure.layers[p.layer]?.visible);
    if (!visible) continue;

    // every(...) is true only if the test holds for every member: here,
    // all within half a pixel of the first one.
    const satisfied = pts.every((p) =>
      Math.hypot(p.x - pts[0].x, p.y - pts[0].y) <= EPS
    );

    drawPlusMark(pts[0].x, pts[0].y, satisfied ? "#000000" : "#d00000");
  }

} // end drawWeldMarks


function drawPlusMark(x, y, color) {
  ctx.save();
  ctx.beginPath();
  ctx.arc(x, y, MARK_RADIUS, 0, Math.PI * 2);
  ctx.fillStyle = "white";
  ctx.fill();
  ctx.lineWidth = 2;
  ctx.strokeStyle = color;
  ctx.beginPath();
```

```
  ctx.moveTo(x - MARK_ARM, y); ctx.lineTo(x + MARK_ARM, y);
  ctx.moveTo(x, y - MARK_ARM); ctx.lineTo(x, y + MARK_ARM);
  ctx.stroke();
  ctx.restore();
} // end drawPlusMark

// end figureWelds.js
```

figuresCommands.js

ui/figures/figuresCommands.js · 218 lines

COMMANDS PANEL AND CAPTION MENU

← Contents

```
/* figuresCommands.js
   ============================================================
   Figures Tab -- Commands Button and Caption Bar
   ============================================================
   Role:
     Owns the "Figures Commands" side panel and the caption bar
     (title plus the "v" menu) for a figure subtab.

   Architectural rules:
     * Does NOT own lifecycle (init/restore/save). That is figures.js.
     * Does NOT build subtabs or switch them. That is figuresNav.js.

   Caption menu, top to bottom:
     Help, Notes          the shared items every tab has; grayed when
                          there is nothing to show
     Hit radius (N px)... how close a points-mode click must be
     Delete selected point   points mode, with a disc selected
     Break apart          Phase 7c, not in points mode
     Show Script, Copy, Archive, Reset    grayed: not built yet, shown
                                          so the menu has its final shape

   The caption bar itself also carries the Line and Parabola buttons
   and the active layer's name and Show checkbox. Those are added by
   addCaptionTools() in figuresNav.js after setFiguresCaption() has
   built the bar.

   Exports:
     wireFiguresCommandsButton()
     setFiguresCaption(title, withMenu)
     getFiguresCaptionMenuItems(title)
   ============================================================ */

import { setCaptionBar }            from "/ui/caption.js";
import { menuManager }              from "/ui/menuManager.js";
import { makeHelpItem, makeNotesItem } from "/ui/menuCmds.js";
import {
  setCommandsButtonLabel,
  setCommandsButtonHandler,
  showCommandsOffcanvas,
  formatRebuildReportShared
}                                   from "/ui/uiUtilities.js";
import { nodeRebuildAndValidateManifests } from "/ui/nodeLayer.js";
import { openHelpHomeOverlay }      from "/ui/help.js";
import { uiState }                  from "/ui/uiState.js";
import { redrawFigure, canBreakApart, breakApartLayer } from "./figureLayers.js";
import { deletePoint }              from "./figurePoints.js";


/* ============================================================
   wireFiguresCommandsButton()
   ------------------------------------------------------------
   The Commands button at the top left opens a slide-in panel
   ("offcanvas", a Bootstrap term) with two buttons:
     Rebuild & Validate  asks the Node service to rebuild every
                         manifest, then reloads this tab
     Help                closes the panel and opens the help overlay
   ============================================================ */
export function wireFiguresCommandsButton() {
```

```
59
60    setCommandsButtonHandler(() => {
61      showCommandsOffcanvas({
62        title: "Figures Commands",
63        // buildBody is called with the panel's body element once the
64        // panel exists; it fills the body and wires the buttons.
65        buildBody(offcanvasBodyEl) {
66
67          offcanvasBodyEl.innerHTML = `
68            <div class="cmdButtonRow">
69              <button id="figuresRebuildValidateButton" class="cmdButton" type="button">
70                Rebuild &amp;amp; Validate
71              </button>
72            </div>
73            <div class="cmdButtonRow">
74              <button id="figuresHelpButton" class="cmdButton" type="button">
75                Help
76              </button>
77            </div>
78            <div class="buttonSeparator"></div>
79            <div id="figuresRebuildReport"></div>
80          `;
81
82          document.getElementById("figuresRebuildValidateButton")
83            .addEventListener("click", async () => {
84              const out = document.getElementById("figuresRebuildReport");
85              out.textContent = "Running Global Rebuild...";
86
87              const report = await nodeRebuildAndValidateManifests();
88
89              // import() inside a function loads the module only when it is
90              // needed, here only after a rebuild. It also avoids an import
91              // loop at load time (figures.js imports this file).
92              const { syncSystemStateAfterRebuild } = await import("/ui/uiUtilities.js");
93              await syncSystemStateAfterRebuild();
94
95              const { initFiguresTab } = await import("./figures.js");
96              await initFiguresTab(false);
97
98              out.textContent = formatRebuildReportShared(report);
99            });
100
101         document.getElementById("figuresHelpButton")
102           .addEventListener("click", () => {
103             const closeBtn = document.querySelector('[data-bs-dismiss="offcanvas"]');
104             if (closeBtn) closeBtn.click();
105             openHelpHomeOverlay();
106           });
107       }
108     });
109   });
110
111 } // end wireFiguresCommandsButton
112
113
114 /* ============================================================
115    setFiguresCaption(title, withMenu)
116    ------------------------------------------------------------
117    title    - caption text (default "Figures")
118    withMenu - true on a figure subtab: shows the "v" menu
119
120    The menu items are built fresh each time the "v" is pressed
121    (getFiguresCaptionMenuItems below), so a grayed or enabled item
122    always reflects the moment it is opened -- for instance, "Delete
123    selected point" is live only if a disc is selected right then.
```

```
   ============================================================ */
export function setFiguresCaption(title = "Figures", withMenu = false) {

  setCommandsButtonLabel("Figures Commands");

  setCaptionBar({
    targetId: "caption",
    title,
    // "a ? b : c" is if-else as an expression: a function on a figure
    // subtab, null (no menu button) on the Categories subtab.
    onMenu: withMenu ? async (anchor) => {
      const items = await getFiguresCaptionMenuItems(title);
      menuManager.open(items, anchor);
    } : null
  });

} // end setFiguresCaption


/* ============================================================
   getFiguresCaptionMenuItems(title)
   ------------------------------------------------------------
   Returns the menu as an array of items. Each item is a plain
   object the shared menuManager understands:
     { label, onClick, disabled?, tooltip? }
   ============================================================ */
export async function getFiguresCaptionMenuItems(title) {

  // Not built yet: shown grayed out so the menu has its final shape.
  const later = (label) => ({
    label,
    disabled: true,
    tooltip:  "Not available yet",
    onClick:  () => {}
  });

  // The active figure tab, read when the item is clicked.
  const activeTab = () => uiState.figures.tabs[uiState.figures.activeSubtab];

  // A plain browser prompt box, checked for a sensible number. The
  // label shows the current value because the menu is rebuilt on
  // every opening.
  const hitRadiusItem = {
    label:   "Hit radius (" + uiState.figures.hitRadius + " px)...",
    tooltip: "How close a click must be to pick a point in points mode",
    onClick: () => {
      const answer = window.prompt(
        "Hit radius in canvas pixels (1 to 50):", String(uiState.figures.hitRadius));
      if (answer === null) return;             // Cancel pressed
      const n = Number(answer);
      if (Number.isFinite(n) && n >= 1 && n <= 50) uiState.figures.hitRadius = n;
    }
  };

  // Usable only in points mode with a disc selected (the menu is rebuilt
  // each time it opens, so this is read fresh).
  const ts0 = activeTab();
  const deletePointItem = {
    label:    "Delete selected point",
    // "!=" (not "!==") on purpose: true for both null and undefined.
    disabled: !(ts0 && ts0.mode === "points" && ts0.selectedPoint != null),
    tooltip:  "Removes the selected disc (points mode). Its number is never reused.",
    onClick: () => {
      const ts = activeTab();
      if (!ts || !ts.figure || ts.selectedPoint == null) return;
```

```
189         deletePoint(ts.figure, ts.selectedPoint);   // also drops it from any weld
190         ts.selectedPoint = null;
191         redrawFigure(uiState.figures.activeSubtab);
192       }
193     };
194
195     // The active layer's object comes apart into separate layers (Phase 7c).
196     // Like Insert, not offered in points mode.
197     const breakApartItem = {
198       label:    "Break apart",
199       disabled: !(ts0 && ts0.mode !== "points" && canBreakApart(uiState.figures.activeSubtab)),
200       tooltip:  "Replaces the active object with its pieces, each on its own layer. The original is
hidden.",
201       onClick: () => breakApartLayer(uiState.figures.activeSubtab)
202     };
203
204     return [
205       await makeHelpItem("figures", null),
206       await makeNotesItem(null, title),
207       hitRadiusItem,
208       deletePointItem,
209       breakApartItem,
210       later("Show Script"),
211       later("Copy"),
212       later("Archive"),
213       later("Reset")
214     ];
215
216   } // end getFiguresCaptionMenuItems
217
218   // end figuresCommands.js
```

figuresDisplay.js

ui/figures/figuresDisplay.js · 123 lines

THE SAVED-FIGURE CATEGORY LIST

← Contents

```
/* figuresDisplay.js
   ============================================================
   Figures Tab -- Category Loading and Figure Display
   ============================================================
   Role:
     Owns everything related to loading the category list and
     loading individual figures.

   Architectural rules:
     * Does NOT build or activate subtabs. That is figuresNav.js.
     * Does NOT build the caption bar. That is figuresCommands.js.
     * Does NOT build figures. Saved figures open in Phase 10; until then
       a category item only reports that it cannot be opened yet.
     * Reads and writes figuresState.js via getters and setters.

   Data formats:
     directoryRegistry.json -- plain array of category name strings
     manifest.json          -- plain array of entry objects

   Where the files are:
     /figures/directoryRegistry.json        the list of categories
     /figures/<category>/manifest.json      the figures in one category,
                                            each { title, filename, ... }
     Both are kept by the local Node service (port 5174);
     node/createFigureService.mjs adds a category to the list. Until Save As
     exists (Phase 10) there are no saved figures, so the Categories
     subtab shows "No saved figures yet".

   Exports:
     loadFiguresCategories()
     loadFigure(filename, categoryName, title)
   ============================================================ */

import { renderCategories }    from "/ui/categories.js";
import { setFiguresRegistry }  from "./figuresState.js";


/* ============================================================
   loadFiguresCategories()
   ------------------------------------------------------------
   Reads the category list, then each category's manifest, and
   hands the result to the shared renderCategories() (ui/categories.js),
   which draws the same accordion list the other tabs show.

   "async" and "await": fetch() asks the local web server for a file
   and returns at once with a promise of the answer. "await" pauses
   this function (not the browser) until the answer arrives, so the
   code reads top to bottom like ordinary sequential code.
   ============================================================ */
export async function loadFiguresCategories() {

  try {
    const res = await fetch("/figures/directoryRegistry.json");
    if (!res.ok) throw new Error("Failed to load directoryRegistry");
    const registry = await res.json();

    setFiguresRegistry(registry);

```

```
    const categories = [];

    for (const categoryName of registry) {      // "for ... of" walks an array's values
      const dirPath = `/figures/${categoryName}`; // backquotes: ${...} is filled in

      // A missing or broken manifest skips that one category; it does
      // not stop the rest from loading.
      try {
        const manRes = await fetch(`${dirPath}/manifest.json`);
        if (!manRes.ok) continue;

        const manifest = await manRes.json();
        const items    = [];

        for (const fig of manifest) {
          const { title, filename } = fig;       // copies fig.title and fig.filename into two names
          items.push({
            name:        title,
            hasSubitems: false,
            // An arrow function: a small function written in place. It
            // remembers filename, categoryName and title from this
            // loop pass, so each item opens its own figure.
            onClick:     () => loadFigure(filename, categoryName, title)
          });
        }

        categories.push({ title: categoryName, items });

      } catch (err) {
        console.warn(`Failed to load manifest for ${categoryName}`, err);
      }
    }

    if (categories.length === 0) {
      document.getElementById("text").innerHTML =
        "<p style='padding:12px'>No saved figures yet. " +
        "Use Create Figure to start one.</p>";
    } else {
      renderCategories("text", categories);
    }

  } catch (e) {
    console.error("Error loading figures categories:", e);
    document.getElementById("text").innerHTML = "Error loading categories.";
  }

} // end loadFiguresCategories


/* ============================================================
   loadFigure(filename, categoryName, title)
   ------------------------------------------------------------
   Opening a saved figure arrives with Save As and reload
   (Phase 10). Until then there is nothing to open.

   When it is built, this will read the saved figure (plain data,
   see figureModel.js), open a new figure subtab for it, and rebuild
   one live instance per layer from each layer's id and params, the
   same way insertLayer() in figureLayers.js builds one today.
   ============================================================ */
export async function loadFigure(filename, categoryName, title) {

  console.warn(`loadFigure: saved figures cannot be opened yet (${categoryName}/${filename})`);

} // end loadFigure
```

figuresState.js

ui/figures/figuresState.js · 84 lines

NAVIGATION BOOKKEEPING

← Contents

```
/* figuresState.js
   ============================================================
   Figures Tab -- Shared Module State
   ============================================================
   Role:
     Owns the module-level state that tracks the Figures tab's
     current navigation position and registry cache.

   Design rules:
     * This file contains NO logic -- only variable declarations
       and their getters/setters.
     * No imports. Must remain a pure data store with zero
       dependencies to avoid circular references.
     * All mutation goes through setter functions for debugging
       and validation.

   Variable glossary:
     figuresRegistry -- Registry data shaped as plain array of
                        category name strings.
                        Null when not yet loaded or cleared.
     currentTabId    -- Currently active tab ID (e.g., "tab-categories"
                        or "tab-figureId").
                        Null when not initialized.

   Where this sits next to uiState:
     These two values are navigation bookkeeping for this module
     only. Everything that describes a figure (its layers, discs,
     welds, mode, selection) lives in uiState.figures, where every
     tab can see it and where it survives a tab switch. If a value
     here and a value in uiState ever seem to disagree, uiState is
     the one that counts: figuresNav.js sets both together.

   Who reads and writes:
     setFiguresRegistry   figuresDisplay.js, after loading the list
                          of saved-figure categories
     setCurrentTabId      figuresNav.js, on every subtab switch
     resetFiguresState    initialization (not called by Figures
                          code at present; kept for the pattern
                          the other tabs follow)
   ============================================================ */

// "let" at the top of a module: one copy for the whole program,
// private to this file. Other files can only reach these through
// the exported functions below. This is how the file keeps its
// "all mutation goes through setters" rule.
let figuresRegistry = null;
let currentTabId    = null;


/* ============================================================
   Getters
   ============================================================ */
export function getFiguresRegistry() {
  return figuresRegistry;
}

export function getCurrentTabId() {
  return currentTabId;
```

```
}


/* ============================================================
   Setters
   ============================================================ */
export function setFiguresRegistry(registry) {
  figuresRegistry = registry;
}

export function setCurrentTabId(tabId) {
  currentTabId = tabId;
}


/* ============================================================
   Reset (used by init)
   ============================================================
   NOTE: Does NOT clear figuresRegistry -- that cache persists
   across navigation and is only cleared by explicit cache
   invalidation (e.g. after rebuild). This function only
   resets the navigation pointer.
   ============================================================ */
export function resetFiguresState() {
  currentTabId = null;
}
```

interactor.js

ui/interactor.js · 311 lines

APPENDIX. HANDLE DRAGGING, SHARED WITH THE DRAW TAB (NOT RE-COMMENTED)

← Contents

```
/* ============================================================
   interactor.js â€“ Unified Interaction Manager
   ============================================================ */
import { canvasLayerManager } from "./canvasLayerManager.js";
import { uiState } from "/ui/uiState.js";
import { applyTransform, inverseTransform } from "/draw/drawTransform.js";

export const PointPickerPresets = {
    handleRadius: 5,        // Normal size
    hitTolerance: 10,       // Grab tolerance
    idleColor: "#C91A09",   // LEGO Bright Red
    activeColor: "#FE8A18"  // LEGO Bright Orange
};

let deactivateCurrent = null;
let renderCurrent     = null;   // redraws the armed object's handles, or null

/* ============================================================
   DRAW SIDE: Coordination Logic
   ============================================================ */

export function armInteractor(instance) {
    // 1. Ensure any existing interactor is fully shut down
    disarmInteractor();

    const pts = instance?.params?.points;
    if (!pts) {
        // Not all objects have points, this is fine.
        return;
    }

    // Disable navigation overlays that might interfere with dragging
    const nav = document.getElementById("next-prev-overlay");
    if (nav) nav.style.pointerEvents = "none";

    // 2. Activate the picker and store the cleanup function
    deactivateCurrent = activatePointPicker(instance.params.points, {
        uiKey: instance.id,
        // Figures: the layer's transform. Handles are stored in object space
        // and shown, hit and dragged in canvas space. Absent (the Draw tab)
        // means the identity.
        getTransform: instance.getTransform,
        // Figures: true when a press at (x, y) belongs to the centroid, which
        // wins over a handle sitting under it.
        yieldTo: instance.yieldTo,
        // Figures, Phase 8: snapshot the layer's welded discs just before a
        // handle moves, and let them follow just after. Undefined (the
        // Draw tab, or a layer with no welds) means moveHandle below does
        // nothing extra.
        beforeMove: instance.beforeMove,
        afterMove:  instance.afterMove,
        onUpdate: () => {
            // Safety: Only proceed if this instance is still the active one
            if (!deactivateCurrent) return;

            // Sync script-internal state
            if (instance.update) {
                instance.update(instance.params);
```

```
            }

            // Trigger engine redraw via uiState bridge OR instance callback
            // Priority:
            // 1. instance.redrawHandler (e.g. Figures tab)
            // 2. uiState.draw (Legacy Draw tab)
            // 3. instance.draw() (Fallback)

            if (typeof instance.redrawHandler === "function") {
                instance.redrawHandler();
            } else {
                const activeTabId = uiState.draw.activeSubtab;
                const activeTab = uiState.draw.tabs[activeTabId];

                if (activeTab && activeTab.redrawHandler) {
                    activeTab.redrawHandler();
                } else if (instance.draw) {
                    instance.draw();
                }
            }
        }
    });
}

// Redraws the handles of the armed object. For code that changes where they
// are without going through the interactor (the Figures centroid moving or
// turning a whole layer).
export function refreshInteractor() {
    if (renderCurrent) renderCurrent();
}

export function disarmInteractor() {
    // 1. Shut down the listeners and clear the cleanup reference
    if (deactivateCurrent) {
        deactivateCurrent();
        deactivateCurrent = null;
    }

    // 2. Hard-reset the interaction canvas layer
    const canvas = canvasLayerManager.get("interaction");
    if (canvas) {
        const ctx = canvas.getContext("2d");
        // Wipe all pixels
        ctx.clearRect(0, 0, canvas.width, canvas.height);
        // Hide the layer so it cannot capture events or show "ghosts"
        canvas.style.display = "none";
        canvas.style.pointerEvents = "none";
    }
}

/* ============================================================
   CORE PICKER LOGIC
   ============================================================ */

function activatePointPicker(points, config = {}) {
    const settings = { ...PointPickerPresets, ...config };
    const canvas = canvasLayerManager.get("interaction");
    const ctx = canvas.getContext("2d");

    // Sync dimensions with the main drawing canvas
    const mainCanvas = window.drawCanvas;
    if (mainCanvas) {
        canvas.width = mainCanvas.width;
        canvas.height = mainCanvas.height;
        canvas.style.display = "block";
```

```
        canvas.style.pointerEvents = "auto";
    }

    let draggedIndex  = null;
    let hoverIndex    = null;
    let selectedIndex = null;   // the handle the arrow keys nudge

    // Use AbortController for clean, one-shot listener removal
    const controller = new AbortController();
    const { signal } = controller;

    function getCanvasCoords(event) {
        const rect = canvas.getBoundingClientRect();
        const scaleX = canvas.width / rect.width;
        const scaleY = canvas.height / rect.height;
        return {
            x: (event.clientX - rect.left) * scaleX,
            y: (event.clientY - rect.top) * scaleY
        };
    }

    // Handles live in object space. These two turn them into canvas space (to
    // show and hit them) and back (to store a move). With no transform both
    // return the point unchanged.
    function toCanvas(pt) {
        const t = settings.getTransform ? settings.getTransform() : null;
        return applyTransform(t, pt);
    }
    function toObject(x, y) {
        const t = settings.getTransform ? settings.getTransform() : null;
        return inverseTransform(t, x, y);
    }

    // The one place a handle is moved, by drag or by arrow key. (x, y) is a
    // canvas position. Anything that must follow a handle (welds, Phase 8)
    // hooks in here: beforeMove sees the layer's discs before the move,
    // afterMove after onUpdate has synced and redrawn it.
    function moveHandle(i, x, y) {
        if (settings.beforeMove) settings.beforeMove();
        const o = toObject(x, y);
        points[i].x = o.x;
        points[i].y = o.y;
        if (settings.onUpdate) settings.onUpdate();
        if (settings.afterMove) settings.afterMove();
    }

    function render() {
        // CRITICAL SAFETY: If the interactor is disarmed, the signal is aborted.
        // Returning here prevents the "ghost draw" when UI controls are touched.
        if (signal.aborted) return;

        ctx.clearRect(0, 0, canvas.width, canvas.height);

        points.forEach((objPt, i) => {
            const pt = toCanvas(objPt);
            const isActive = (i === draggedIndex || i === hoverIndex || i === selectedIndex);

            ctx.beginPath();
            ctx.arc(pt.x, pt.y, settings.handleRadius, 0, Math.PI * 2);

            // Red dot normally, Orange when active
            ctx.fillStyle = isActive ? settings.activeColor : settings.idleColor;
            ctx.fill();

            // High-contrast stroke for visibility against dark backgrounds
```

```
            ctx.strokeStyle = "white";
            ctx.lineWidth = 1.5;
            ctx.stroke();

            // The selected handle (the one the arrow keys move) gets an outer ring.
            if (i === selectedIndex) {
                ctx.beginPath();
                ctx.arc(pt.x, pt.y, settings.handleRadius + 4, 0, Math.PI * 2);
                ctx.strokeStyle = "black";
                ctx.lineWidth = 1.5;
                ctx.stroke();
            }
        });
    }

    const handleMouseDown = (e) => {
        // Left button only. A right-click (Figures: weld) must not grab a
        // handle, or the smallest hand jitter drags it to the cursor.
        if (e.button !== 0) return;

        const mouse = getCanvasCoords(e);

        // The centroid (Figures) outranks a handle under it. Let it have the press.
        if (settings.yieldTo && settings.yieldTo(mouse.x, mouse.y)) {
            draggedIndex  = null;
            if (selectedIndex !== null) {
                selectedIndex = null;
                render();
            }
            return;
        }

        for (let i = 0; i < points.length; i++) {
            const pt = toCanvas(points[i]);
            const dist = Math.sqrt((mouse.x - pt.x) ** 2 + (mouse.y - pt.y) ** 2);
            if (dist < settings.hitTolerance) {
                draggedIndex  = i;
                selectedIndex = i;
                render();
                return;
            }
        }
        // A press anywhere else clears the selection.
        if (selectedIndex !== null) {
            selectedIndex = null;
            render();
        }
    };

    const handleMouseMove = (e) => {
        const mouse = getCanvasCoords(e);
        let currentHover = null;

        for (let i = 0; i < points.length; i++) {
            const pt = toCanvas(points[i]);
            const dist = Math.sqrt((mouse.x - pt.x) ** 2 + (mouse.y - pt.y) ** 2);
            if (dist < settings.hitTolerance) {
                currentHover = i;
                break;
            }
        }

        if (currentHover !== hoverIndex || draggedIndex !== null) {
            hoverIndex = currentHover;
            if (draggedIndex !== null) {
```

```
                moveHandle(draggedIndex, mouse.x, mouse.y);
            }
            render();
        }
    };

    const handleMouseUp = () => {
        draggedIndex = null;
        render();
    };

    // Arrow keys nudge the selected handle 1 px, or 10 px with Shift. Ignored
    // while the keyboard is in a slider, list or text box.
    const handleKeyDown = (e) => {
        if (selectedIndex === null) return;

        const tag = (e.target && e.target.tagName) || "";
        if (tag === "INPUT" || tag === "SELECT" || tag === "TEXTAREA") return;

        const step = e.shiftKey ? 10 : 1;
        let dx = 0;
        let dy = 0;
        switch (e.key) {
            case "ArrowLeft":  dx = -step; break;
            case "ArrowRight": dx =  step; break;
            case "ArrowUp":    dy = -step; break;
            case "ArrowDown":  dy =  step; break;
            default: return;
        }

        e.preventDefault();
        const pt = toCanvas(points[selectedIndex]);
        moveHandle(selectedIndex, pt.x + dx, pt.y + dy);
        render();
    };

    // Attach listeners to window to ensure we don't lose the drag if mouse leaves canvas
    window.addEventListener("keydown",   handleKeyDown,   { signal });
    window.addEventListener("mousedown", handleMouseDown, { signal });
    window.addEventListener("mousemove", handleMouseMove, { signal });
    window.addEventListener("mouseup",   handleMouseUp,   { signal });

    // Initial render call to show dots immediately
    render();
    renderCurrent = render;

    // The cleanup function returned to 'deactivateCurrent'
    return () => {
        renderCurrent = null;
        controller.abort();
        ctx.clearRect(0, 0, canvas.width, canvas.height);
        canvas.style.display = "none";
    };
}

// Global Exports
window.armInteractor = armInteractor;
window.disarmInteractor = disarmInteractor;
```